Lab Manual

Data Mining

DS-303

By

Wishaq Akbar **2023-BSAI-041**

Submitted to
Mr. Arham

Bachelor of Science in Artificial Intelligence
DEPARTMENT OF COMPUTER SCIENCE

Content

Lab 1: Introduction to Data Mining

1.1 Objectives

- Understand the concept and significance of data mining
- Identify the types of data and knowledge representations
- Differentiate data mining from related disciplines
- Explore real-world applications of data mining

1.2 Introduction

Data mining is the process of discovering patterns, correlations, anomalies, and useful insights from large datasets. It combines techniques from statistics, machine learning, database management, and pattern recognition to extract knowledge from raw data. The explosive growth in data from business transactions, social media, sensors, and the web has made data mining one of the most critical fields in modern computing.

Data mining is often referred to as Knowledge Discovery in Databases (KDD). The KDD process encompasses data selection, preprocessing, transformation, mining, and interpretation/evaluation of discovered patterns.

1.3 Background Theory

1.3.1 What is Data Mining?

Data mining extracts implicit, previously unknown, and potentially useful information from data. Unlike simple queries, data mining discovers patterns not explicitly stated in the data. Key characteristics include:

- Large datasets (Big Data environments)
- Automatic or semi-automatic discovery
- Useful and actionable patterns
- Support for decision-making

1.3.2 KDD vs Data Mining

The KDD process is the overall pipeline for turning raw data into knowledge:

Stage	Description
Data Selection	Choose relevant data from databases
Data Preprocessing	Clean noise, handle missing values
Data Transformation	Normalize, aggregate, or encode
Data Mining	Apply algorithms to find patterns
Interpretation/Evaluation	Validate and present knowledge

1.3.3 Types of Data

- Relational data (tables in databases)

- Transactional data (purchase records, logs)
- Time-series data (stock prices, sensor readings)
- Spatial and spatiotemporal data (GIS, maps)
- Text data (documents, emails, social media)
- Multimedia data (images, audio, video)
- Web data (hyperlinks, clickstreams)

1.3.4 Types of Knowledge Discovered

- Association rules: If A then B (e.g., bread implies butter)
- Classification: Assigning categories to instances
- Clustering: Grouping similar objects together
- Regression: Predicting numerical values
- Anomaly detection: Identifying unusual patterns
- Sequential patterns: Ordered event patterns over time

1.3.5 Data Mining vs Related Fields

Field	Focus	Relation to Data Mining
Statistics	Mathematical analysis of data	Provides theoretical foundation
Machine Learning	Algorithms that learn from data	Provides core algorithms
Database Systems	Data storage and retrieval	Provides data access
Data Warehousing	Integrated, historical data storage	Provides data sources
AI	Intelligent behaviour simulation	Shares goals and methods

1.4 Applications of Data Mining

1.4.1 Business and Retail

- Market basket analysis (Amazon customers also bought)
- Customer segmentation and churn prediction
- Demand forecasting and inventory management

1.4.2 Healthcare

- Disease prediction and early diagnosis
- Drug interaction analysis
- Patient readmission risk scoring

1.4.3 Finance

- Credit scoring and loan default prediction
- Fraud detection in transactions
- Algorithmic stock trading signals

1.4.4 Telecommunications

- Call detail record analysis
- Network intrusion detection
- Customer retention programs

1.5 Lab Activity: Exploring a Dataset

1.5.1 Tools Required

- Python with pandas, numpy, matplotlib installed
- Jupyter Notebook or VS Code with Jupyter extension

1.5.2 Dataset

Use the built-in Iris dataset from sklearn, or download the Titanic dataset from Kaggle.

1.5.3 Step-by-Step Instructions

1. Open Jupyter Notebook and create a new Python 3 notebook.
2. Import required libraries:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
```

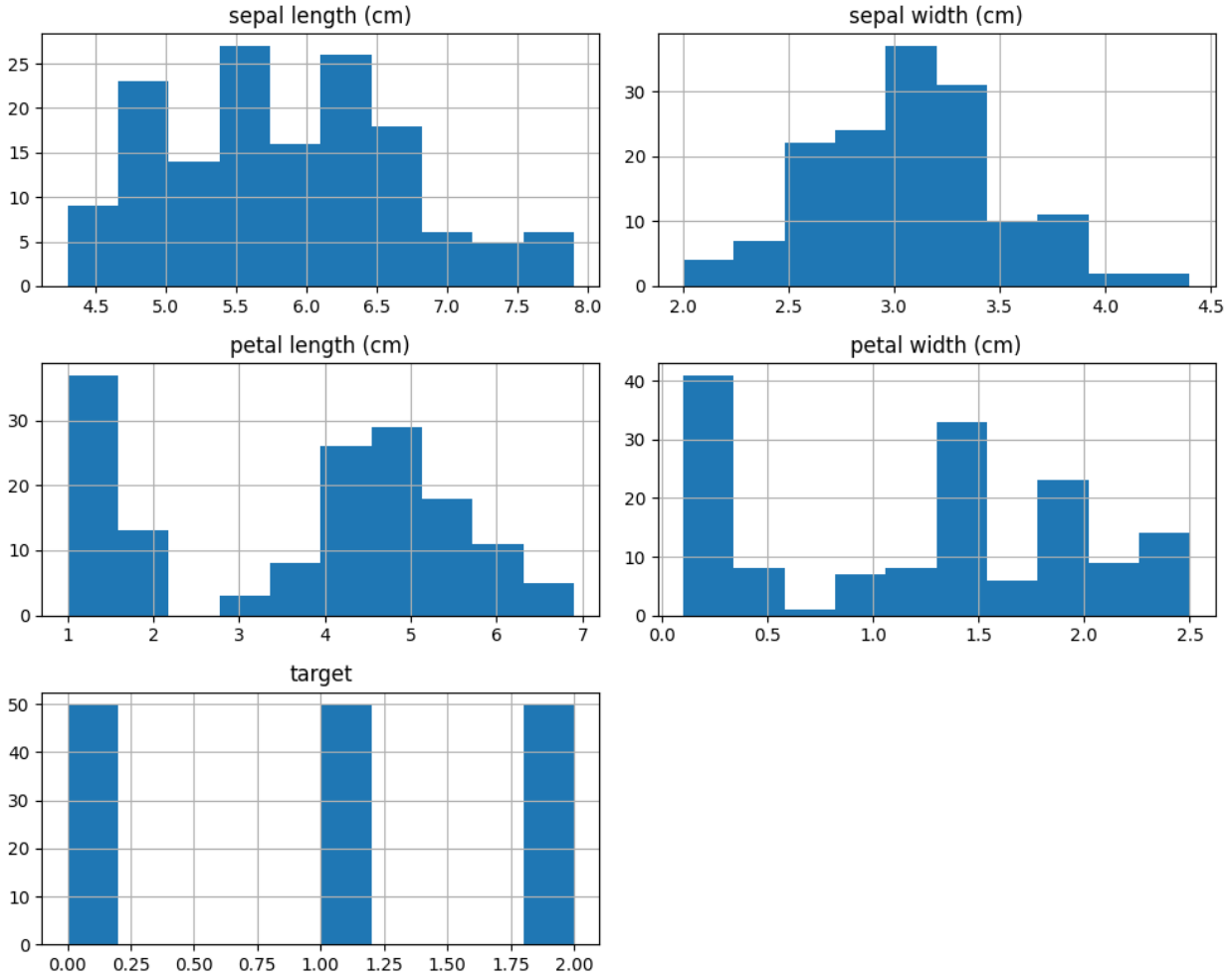
3. Load and inspect the dataset:

```
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target
print(df.head())
print(df.describe())
print(df.info())
```

```
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    int64
dtypes: float64(4), int64(1)
```

4. Visualize the data:

```
df.hist(figsize=(10, 8))
plt.tight_layout()
plt.show()
```



5. Identify class distribution:
   `print(df['target'].value_counts())`

```
target
0    50
1    50
2    50
Name: count, dtype: int64
```

1.5.4 Observations

Record your answers to the following questions in your lab notebook:

6. How many instances and features does the dataset have?
7. What types of features are present (categorical, numerical, ordinal)?
8. Are there any visible patterns in the histograms?
9. What mining task would be most appropriate for this data?

1.6 Summary

Data mining is a multidisciplinary field that extracts actionable knowledge from large datasets. It forms the mining stage within the broader KDD process. Understanding data types, knowledge representations, and application domains is fundamental to selecting appropriate mining techniques in subsequent labs.

Lab 2: CRISP-DM Process

2.1 Objectives

- Understand the CRISP-DM lifecycle and its six phases
- Apply CRISP-DM to a real-world problem scenario
- Produce a project plan document following CRISP-DM

2.2 Introduction

CRISP-DM (Cross-Industry Standard Process for Data Mining) is the most widely used methodology for data mining projects. Developed in the late 1990s by a consortium of companies, it provides a structured, iterative, and non-proprietary framework that guides practitioners through the full data mining lifecycle.

CRISP-DM is not prescriptive about tools or techniques. It is process-oriented, ensuring thoroughness and repeatability regardless of the domain or technology stack.

2.3 The Six Phases of CRISP-DM

2.3.1 Phase 1: Business Understanding

The first phase focuses on understanding project requirements and objectives from a business perspective, then converting this knowledge into a data mining problem definition.

- Define business objectives (e.g., reduce customer churn by 15%)
- Assess the current situation: resources, constraints, risks
- Define data mining goals (e.g., build a classifier with >85% accuracy)
- Produce a project plan with tasks, timelines, and success criteria

2.3.2 Phase 2: Data Understanding

This phase involves collecting initial data, familiarizing with the data, and identifying data quality problems.

- Collect initial data from relevant sources
- Describe data: format, number of records, field meanings
- Explore data: distributions, relationships, outliers
- Verify data quality: completeness, consistency, accuracy

2.3.3 Phase 3: Data Preparation

Data preparation covers all activities to construct the final dataset from raw data. This is typically the most time-consuming phase (60-80% of project time).

- Select relevant data subsets
- Clean data: handle missing values, remove duplicates, fix inconsistencies
- Construct new features (feature engineering)
- Integrate data from multiple sources
- Format data to meet tool requirements

2.3.4 Phase 4: Modelling

Various modelling techniques are selected and applied, and their parameters are calibrated to optimal values.

- Select modelling technique(s) based on mining goals
- Design test setup (train/validation/test splits, cross-validation)
- Build the model(s) using training data
- Assess model quality and tune parameters

2.3.5 Phase 5: Evaluation

The model is evaluated more thoroughly against business objectives before final deployment.

- Evaluate results against business success criteria
- Review the process for overlooked steps or issues
- Determine next steps: deploy, iterate, or abandon

2.3.6 Phase 6: Deployment

Knowledge gained must be organized and presented in a way the stakeholder can use.

- Plan deployment: integrate into production systems
- Plan monitoring and maintenance
- Produce final report and presentation
- Review project: lessons learned

2.4 CRISP-DM Iteration

CRISP-DM is inherently iterative. Findings in later phases often require revisiting earlier ones. For example, evaluation results may indicate a need to return to data preparation to engineer better features.

2.5 CRISP-DM Phase Summary Table

Phase	Key Outputs	Typical Time (%)
Business Understanding	Project plan, mining goal	5-10%
Data Understanding	Data report, quality assessment	10-15%
Data Preparation	Cleaned dataset, feature set	50-70%
Modelling	Model, parameter settings	10-15%
Evaluation	Evaluation report, decision	5-10%
Deployment	Deployment plan, final report	5%

2.6 Summary

CRISP-DM provides a robust, iterative framework for structuring data mining projects. Adherence to its phases ensures that business needs remain central, data is properly understood and prepared, models are rigorously evaluated, and results are deployed effectively. All subsequent labs in this manual operate within the CRISP-DM framework.

Lab 3: Handling Missing Values, Duplicates & Noise

3.1 Objectives

- Detect and handle missing values using various imputation strategies
- Identify and remove duplicate records
- Detect and smooth noisy data

3.2 Introduction

Real-world data is rarely clean. Data quality issues including missing values, duplicate records, and noise can significantly degrade the performance of data mining models. Data cleaning is therefore a critical preprocessing step that occupies the majority of a data scientist's time.

3.3 Missing Values

3.3.1 Types of Missingness

Type	Definition	Example
MCAR	Missing Completely at Random - no pattern	Random sensor failures
MAR	Missing at Random - missingness depends on other observed data	Income missing more for younger respondents
MNAR	Missing Not at Random - missingness depends on the missing value itself	High earners skip income question

3.3.2 Handling Strategies

- Deletion: Remove rows/columns with missing values (listwise or pairwise)
- Mean/Median/Mode imputation: Replace with central tendency measures
- Forward/Backward fill: Use adjacent values (for time-series data)
- KNN imputation: Use values from k-nearest neighbours
- Regression imputation: Predict missing values from other features
- Multiple Imputation: Generate multiple plausible values using statistical models

3.3.3 Python Implementation

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer, KNNImputer

# Load dataset
df = pd.read_csv('data.csv')

# Check missing values
```

```
print(df.isnull().sum())
print(df.isnull().mean() * 100)  # percentage missing

# Mean imputation
imputer = SimpleImputer(strategy='mean')
df[['Age', 'Salary']] = imputer.fit_transform(df[['Age',
'Salary']])

# Median imputation
imputer_med = SimpleImputer(strategy='median')
df[['Income']] = imputer_med.fit_transform(df[['Income']])

# KNN imputation
knn_imputer = KNNImputer(n_neighbors=5)
df_imputed = pd.DataFrame(knn_imputer.fit_transform(df),
columns=df.columns)
```

```
Missing values per column:
Age       2
Salary    2
Income    3
dtype: int64
```

```
Percentage missing:
Age       25.0
Salary    25.0
Income    37.5
dtype: float64
```

```
Imputed DataFrame (Mean/Median):
         Age   Salary   Income
0  25.000000  50000.0  45000.0
1  30.000000  60000.0  48000.0
2  34.666667  55000.0  48000.0
3  40.000000  60000.0  52000.0
4  35.000000  58000.0  48000.0
5  28.000000  60000.0  46000.0
6  34.666667  62000.0  51000.0
7  50.000000  75000.0  48000.0
```

```
Imputed DataFrame (KNN):
    Age   Salary   Income
0  25.0  50000.0  45000.0
1  30.0  60000.0  49000.0
2  31.5  55000.0  48000.0
3  40.0  59000.0  52000.0
4  35.0  58000.0  49000.0
5  28.0  59000.0  46000.0
6  35.0  62000.0  51000.0
7  50.0  75000.0  49000.0
```

3.4 Duplicate Records

3.4.1 Why Duplicates Occur

Duplicates arise from data entry errors, system migrations, merging datasets from multiple sources, or web scraping. They bias model training by over-representing certain records.

3.4.2 Detection and Removal

```
# Detect duplicates
print(df.duplicated().sum())
print(df[df.duplicated()])  # show duplicate rows

# Remove duplicates
```

```
df_clean = df.drop_duplicates()
df_clean = df.drop_duplicates(subset=['CustomerID'])  # key-based
```

```
Number of duplicates: 0
Empty DataFrame
Columns: [Age, Salary, Income]
Index: []
```

3.5 Noisy Data

3.5.1 Sources of Noise

- Faulty data collection instruments
- Data entry errors or OCR mistakes
- Transmission errors in networked systems
- Deliberate outliers or adversarial data

3.5.2 Noise Detection Methods

- Statistical: Z-score, IQR-based outlier detection
- Visual: Box plots, scatter plots, histograms
- Domain knowledge: Values outside physically plausible ranges

3.5.3 Smoothing Techniques

- Binning: Equal-width or equal-frequency binning, then smooth by mean or boundary
- Regression: Fit a regression model and use predicted values
- Clustering: Detect outliers as points not belonging to any cluster
- Moving average: Replace each value with the average of a sliding window

3.5.4 Python Implementation

```
# Z-score outlier detection
from scipy import stats
z_scores = np.abs(stats.zscore(df[['Age', 'Salary']]))
df_clean = df[(z_scores < 3).all(axis=1)]

# IQR method
Q1 = df['Salary'].quantile(0.25)
Q3 = df['Salary'].quantile(0.75)
IQR = Q3 - Q1
df_clean = df[~((df['Salary'] < (Q1 - 1.5 * IQR)) | (df['Salary']
> (Q3 + 1.5 * IQR)))]

# Binning-based smoothing
df['Age_bin'] = pd.cut(df['Age'], bins=5, labels=['Very
Young','Young','Middle','Senior','Old'])
```

```
df['Age_mean_smooth'] =
df.groupby('Age_bin')['Age'].transform('mean')
```

```
df['Age_mean_smooth'] = df.groupby('Age_bin')['Age'].transform('mean')
```

3.6 Lab Exercises

1. Load the Titanic dataset and compute the percentage of missing values for each column.

```
import seaborn as sns
from sklearn.impute import SimpleImputer, KNNImputer

titanic = sns.load_dataset('titanic')

# Compute percentage of missing values
missing_pct = titanic.isnull().mean() * 100
print("Percentage of missing values per column:")
print(missing_pct[missing_pct > 0])
```

```
Percentage of missing values per column:
age            19.865320
embarked        0.224467
deck           77.216611
embark_town     0.224467
dtype: float64
```

2. Apply mean imputation to the Age column and KNN imputation to Fare. Compare results.

```
# Mean Imputation for Age
mean_imputer = SimpleImputer(strategy='mean')
titanic['Age_mean_imputed'] =
mean_imputer.fit_transform(titanic[['age']])

# KNN Imputation for Fare (and Age to provide context for KNN)
knn_imputer = KNNImputer(n_neighbors=5)
# KNN requires numeric columns
numeric_cols = ['age', 'fare', 'pclass', 'sibsp', 'parch']
knn_results = knn_imputer.fit_transform(titanic[numeric_cols])
titanic['Fare_knn_imputed'] = knn_results[:, 1]

# Compare results for rows where Fare was potentially missing (though
Fare is usually complete in this set)
print("\nImputation comparison (First 5 rows):")
```

```
display(titanic[['age', 'Age_mean_imputed', 'fare',
'Fare_knn_imputed']].head())
```

Imputation comparison (First 5 rows):

	age	Age_mean_imputed	fare	Fare_knn_imputed
0	22.0	22.0	7.2500	7.2500
1	38.0	38.0	71.2833	71.2833
2	26.0	26.0	7.9250	7.9250
3	35.0	35.0	53.1000	53.1000
4	35.0	35.0	8.0500	8.0500

3. Identify and remove duplicate passenger records based on Name and Ticket.

```
duplicates = titanic.duplicated().sum()
print(f"\nNumber of duplicate records found: {duplicates}")

titanic_clean = titanic.drop_duplicates()
print(f"Rows after removing duplicates: {len(titanic_clean)}")
```

```
Number of duplicate records found: 107
Rows after removing duplicates: 784
```

4. Use Z-score and IQR to detect outliers in the Fare column. How many outliers does each method find?

```
from scipy import stats

# Z-score
z_scores = np.abs(stats.zscore(titanic['fare']))
z_outliers = (z_scores > 3).sum()

# IQR
Q1 = titanic['fare'].quantile(0.25)
Q3 = titanic['fare'].quantile(0.75)
IQR = Q3 - Q1
iqr_outliers = ((titanic['fare'] < (Q1 - 1.5 * IQR)) | (titanic['fare']
> (Q3 + 1.5 * IQR))).sum()

print(f"\nOutliers in 'Fare':")
print(f"Z-score method found: {z_outliers}")
```

```
print(f"IQR method found: {iqr_outliers}")
```

```
Outliers in 'Fare':
Z-score method found: 20
IQR method found: 116
```

5. Apply equal-frequency binning to Age with 4 bins and smooth by bin mean.

```
titanic['Age_bin'] = pd.qcut(titanic['Age_mean_imputed'], q=4)
titanic['Age_smoothed'] = titanic.groupby('Age_bin',
observed=True)['Age_mean_imputed'].transform('mean')

print("\nAge binning and smoothing results:")
display(titanic[['Age_mean_imputed', 'Age_bin',
'Age_smoothed']].head())
```

Age binning and smoothing results:

	Age_mean_imputed	Age_bin	Age_smoothed
0	22.0	(0.419, 22.0]	14.645758
1	38.0	(35.0, 80.0]	46.979263
2	26.0	(22.0, 29.699]	27.987102
3	35.0	(29.699, 35.0]	32.287611
4	35.0	(29.699, 35.0]	32.287611

3.7 Summary

Data cleaning is a non-negotiable step before mining. Missing values require careful imputation strategies chosen based on the mechanism of missingness. Duplicates must be identified and removed to prevent model bias. Noise must be detected and smoothed to ensure robust pattern discovery.

Lab 4: Normalization, Standardization & Discretization

4.1 Objectives

- Understand why feature scaling is necessary
- Apply min-max normalization, z-score standardization, and decimal scaling
- Perform equal-width and equal-frequency discretization
- Recognize when to use each technique

4.2 Introduction

Many machine learning algorithms are sensitive to the scale of input features. Distance-based algorithms (KNN, SVM, k-means) and gradient-based optimizers perform poorly when features have vastly different ranges. Normalization and standardization bring features to comparable scales. Discretization converts continuous attributes into categorical intervals, enabling certain algorithms such as Apriori and decision trees to work more efficiently.

4.3 Normalization

4.3.1 Min-Max Normalization

Scales data to a fixed range, typically [0, 1]:

```
x_norm = (x - x_min) / (x_max - x_min)
```

Advantages: Preserves original distribution shape; useful for bounded domains.

Disadvantages: Sensitive to outliers; new data outside training range maps outside [0,1].

4.3.2 Decimal Scaling

Divides values by a power of 10 so all values fall within [-1, 1]:

```
x_decimal = x / 10^j  # where j = ceil(log10(max(|x|)))
```

4.3.3 Python Implementation

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age',
'Salary']])
```

	Employee_ID	Years_Experience	Salary	Bonus	Department
0	1001	1.000000	0.422861	5048.438913	HR
1	1002	0.736842	0.750741	7824.441111	Marketing
2	1003	0.368421	0.435322	4840.717218	Marketing
3	1004	0.184211	0.231943	5904.743592	IT
4	1005	0.526316	0.437587	2875.212944	IT

4.4 Standardization

4.4.1 Z-Score Standardization

Transforms data to have zero mean and unit variance:

```
x_std = (x - mean) / std_dev
```

Advantages: Handles outliers better than min-max; suitable for algorithms assuming Gaussian input.
Disadvantages: Loses bounded range property.

4.4.2 Python Implementation

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age',
'Salary']])

# Verify
print(df[['Age', 'Salary']].mean())   # should be ~0
print(df[['Age', 'Salary']].std())    # should be ~1
```

```
Means (should be ~0):
Age       0.0
Salary    0.0
dtype: float64

Standard Deviations (should be ~1):
Age       1.069045
Salary    1.069045
dtype: float64
```

4.5 When to Use Which

Technique	Best Used When	Avoid When
Min-Max	Data has known bounds; neural networks	Outliers are present
Z-Score	Normal-ish distribution; SVM, PCA	Distribution is heavily skewed
Decimal Scaling	Simple quick scaling needed	Precision matters critically

4.6 Discretization

4.6.1 What is Discretization?

Discretization converts continuous numerical attributes into discrete categorical intervals (bins). It is also called binning or quantization.

4.6.2 Equal-Width Binning

Divides the range [min, max] into k intervals of equal width:

```
width = (max - min) / k

# Python: equal-width
df['Age_ew'] = pd.cut(df['Age'], bins=4)
print(df['Age_ew'].value_counts())
```

```
Bin width: 0.85

Value counts for Age bins:
Age_ew
(-1.27, -0.417]    3
(-0.417, 0.434]    3
(0.434, 1.284]     1
(1.284, 2.134]     1
Name: count, dtype: int64
```

	Age	Age_ew
0	-1.266584	(-1.27, -0.417]
1	-0.586539	(-1.27, -0.417]
2	-0.382525	(-0.417, 0.434]
3	0.773551	(0.434, 1.284]
4	0.093506	(-0.417, 0.434]

4.6.3 Equal-Frequency Binning

Ensures each bin has approximately the same number of records:

```
# Python: equal-frequency (quantile-based)
df['Age_ef'] = pd.qcut(df['Age'], q=4,
labels=['Q1','Q2','Q3','Q4'])
print(df['Age_ef'].value_counts())
```

```
Value counts for Equal-Frequency bins:
Age_ef
Q1    2
Q2    2
Q3    2
Q4    2
Name: count, dtype: int64
```

	Age	Age_ef
0	-1.266584	Q1
1	-0.586539	Q2
2	-0.382525	Q2
3	0.773551	Q4
4	0.093506	Q3

4.6.4 Comparison

Method	Equal-Width	Equal-Frequency
Bin boundaries	Fixed intervals	Variable intervals
Data distribution	May create empty bins	Handles skew better
Use case	Uniform distributions	Skewed distributions

4.7 Lab Exercises

1. Load the Boston Housing dataset. Apply min-max normalization to all numeric features. Print the first 5 rows before and after.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# 1. Load the Boston Housing dataset from source
data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep="\\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]

columns = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS',
'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
boston_df = pd.DataFrame(data, columns=columns)
boston_df['MEDV'] = target

print("Original Data (First 5 rows):")
display(boston_df.head())

# Apply Min-Max Normalization
scaler_minmax = MinMaxScaler()
boston_minmax = boston_df.copy()
boston_minmax[columns] =
scaler_minmax.fit_transform(boston_df[columns])

print("\nMin-Max Normalized Data (First 5 rows):")
display(boston_minmax.head())
```

```
Original Data (First 5 rows):
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33	36.2

```
Min-Max Normalized Data (First 5 rows):
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B
0	0.000000	0.18	0.067815	0.0	0.314815	0.577505	0.641607	0.269203	0.000000	0.208015	0.287234	1.000000
1	0.000236	0.00	0.242302	0.0	0.172840	0.547998	0.782698	0.348962	0.043478	0.104962	0.553191	1.000000
2	0.000236	0.00	0.242302	0.0	0.172840	0.694386	0.599382	0.348962	0.043478	0.104962	0.553191	0.989737
3	0.000293	0.00	0.063050	0.0	0.150206	0.658555	0.441813	0.448545	0.086957	0.066794	0.648936	0.994276
4	0.000705	0.00	0.063050	0.0	0.150206	0.687105	0.528321	0.448545	0.086957	0.066794	0.648936	1.000000

2. Apply Z-score standardization to the same dataset. Verify mean is approximately 0 and std is approximately 1 for each column.

```
scaler_std = StandardScaler()
boston_std = boston_df.copy()
boston_std[columns] = scaler_std.fit_transform(boston_df[columns])

print("Verification of Z-score Standardization (Mean and Std):")
stats_check = pd.DataFrame({
    'Mean': boston_std[columns].mean(),
    'Std': boston_std[columns].std()
})
display(stats_check.head())
```

```
Verification of Z-score Standardization (Mean and Std):
```

	Mean	Std
CRIM	-1.123388e-16	1.00099
ZN	7.898820e-17	1.00099
INDUS	2.106352e-16	1.00099
CHAS	-3.510587e-17	1.00099
NOX	-1.965929e-16	1.00099

3. Discretize the MEDV (median home value) column using equal-width binning with k=5. Label the bins appropriately.

```
# 3. Discretize MEDV using equal-width binning (k=5)
boston_df['MEDV_equal_width'] = pd.cut(boston_df['MEDV'], bins=5,
labels=['Very Low', 'Low', 'Medium', 'High', 'Very High'])

# 4. Discretize MEDV using equal-frequency binning (k=5)
boston_df['MEDV_equal_freq'] = pd.qcut(boston_df['MEDV'], q=5,
labels=['Q1', 'Q2', 'Q3', 'Q4', 'Q5'])

print("\nComparison of Bin Frequency Counts:")
print("--- Equal-Width ---")
print(boston_df['MEDV_equal_width'].value_counts().sort_index())
print("\n--- Equal-Frequency ---")
print(boston_df['MEDV_equal_freq'].value_counts().sort_index())
```

```
Comparison of Bin Frequency Counts:
--- Equal-Width ---
MEDV_equal_width
Very Low      77
Low          239
Medium       123
High          36
Very High     31
Name: count, dtype: int64
```

```
--- Equal-Frequency ---
MEDV_equal_freq
Q1    102
Q2    101
Q3    101
Q4    101
Q5    101
Name: count, dtype: int64
```

4. Repeat with equal-frequency binning. Compare the bin frequency counts from both methods using value_counts().
5. Which scaling method would you recommend for a KNN classifier on this dataset? Justify your answer.

4.8 Summary

Feature scaling is essential for many machine learning algorithms. Min-max normalization is suitable for bounded data and neural networks, while z-score standardization is preferred for normally distributed data used in distance-based or linear models. Discretization converts continuous features into categorical ones, enabling algorithms that require nominal inputs.

Lab 5: Market Basket Analysis

5.1 Objectives

- Understand the concept and business value of market basket analysis
- Interpret association rule metrics: support, confidence, and lift
- Manually compute association rule metrics from transaction data
- Apply the mlxtend library to perform market basket analysis

5.2 Introduction

Market basket analysis (MBA) is a data mining technique used to discover purchase patterns in retail transaction data. The goal is to identify sets of products that frequently appear together in customer purchases. Retailers like Amazon, Walmart, and supermarkets use MBA to optimize product placement, design cross-selling promotions, and build recommendation engines.

MBA is based on association rule mining, which finds rules of the form: {Antecedent} => {Consequent}. For example: {bread, butter} => {milk} means customers who buy bread and butter are also likely to buy milk.

5.3 Key Metrics

5.3.1 Support

Support measures how frequently an itemset appears in the dataset:

```
Support(A) = Transactions containing A / Total transactions
Support(A => B) = Transactions containing both A and B / Total
```

5.3.2 Confidence

Confidence measures the conditional probability that a transaction containing A also contains B:

```
Confidence(A => B) = Support(A union B) / Support(A)
```

5.3.3 Lift

Lift measures how much more likely B is purchased when A is purchased, compared to the base rate of B:

```
Lift(A => B) = Confidence(A => B) / Support(B)
```

Interpretation: Lift > 1 means positive correlation; Lift = 1 means independence; Lift < 1 means negative correlation.

5.3.4 Other Metrics

Metric	Formula	Interpretation
Conviction	(1 - Support(B)) / (1 - Confidence(A=>B))	Higher = stronger implication

Leverage	Support(A union B) - Support(A)*Support(B)	>0 = positive dependency
Jaccard	Support(A union B) / (Sup(A)+Sup(B)-Sup(AuB))	Set similarity measure

5.4 Manual Calculation Example

5.4.1 Transaction Database

TID	Items Purchased
T1	Bread, Milk, Butter
T2	Bread, Diapers, Beer, Eggs
T3	Milk, Diapers, Beer, Cola
T4	Bread, Milk, Diapers, Beer
T5	Bread, Milk, Diapers, Cola

5.4.2 Calculations

Total transactions = 5

- Support({Bread}) = 4/5 = 0.80
- Support({Milk}) = 4/5 = 0.80
- Support({Bread, Milk}) = 3/5 = 0.60
- Confidence({Bread} => {Milk}) = 0.60 / 0.80 = 0.75
- Lift({Bread} => {Milk}) = 0.75 / 0.80 = 0.9375

5.5 Python Implementation

5.5.1 Setup

```
pip install mlxtend

import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
```

5.5.2 Code

```
# Transaction data
dataset = [['Bread', 'Milk', 'Butter'],
           ['Bread', 'Diapers', 'Beer', 'Eggs'],
           ['Milk', 'Diapers', 'Beer', 'Cola'],
           ['Bread', 'Milk', 'Diapers', 'Beer'],
           ['Bread', 'Milk', 'Diapers', 'Cola']]
```

```
# Encode transactions
te = TransactionEncoder()
te_array = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_array, columns=te.columns_)

# Find frequent itemsets
frequent_itemsets = apriori(df, min_support=0.6,
use_colnames=True)
print(frequent_itemsets)

# Generate association rules
rules = association_rules(frequent_itemsets, metric='lift',
min_threshold=1.0)
rules = rules.sort_values('lift', ascending=False)
print(rules[['antecedents', 'consequents', 'support',
'confidence', 'lift']])
```

```
   support          itemsets
0      0.6           (Beer)
1      0.8          (Bread)
2      0.8        (Diapers)
3      0.8           (Milk)
4      0.6   (Beer, Diapers)
5      0.6  (Bread, Diapers)
6      0.6     (Bread, Milk)
7      0.6   (Diapers, Milk)
  antecedents consequents  support  confidence  lift
0      (Beer)   (Diapers)      0.6        1.00  1.25
1   (Diapers)      (Beer)      0.6        0.75  1.25
```

5.6 Lab Exercises

1. Using the transaction table in Section 5.4, manually compute: Support({Beer, Diapers}), Confidence({Diapers} => {Beer}), and Lift({Diapers} => {Beer}).

```
import matplotlib.pyplot as plt
import seaborn as sns

# 2. Modify min_support to 0.4
frequent_itemsets_04 = apriori(df, min_support=0.4, use_colnames=True)
print(f"Frequent itemsets at 0.6 support: {len(frequent_itemsets)}")
print(f"Frequent itemsets at 0.4 support: {len(frequent_itemsets_04)}")
```

```
print(f"Difference: {len(frequent_itemsets_04) -
len(frequent_itemsets)} new itemsets discovered.")

# 3. Filter rules: Confidence > 0.8 and Lift > 1.2
all_rules = association_rules(frequent_itemsets_04, metric='lift',
min_threshold=0.1)
filtered_rules = all_rules[(all_rules['confidence'] > 0.8) &
(all_rules['lift'] > 1.2)]
print("\nFiltered Rules (Confidence > 0.8 & Lift > 1.2):")
display(filtered_rules[['antecedents', 'consequents', 'support',
'confidence', 'lift']])
```

```
Frequent itemsets at 0.6 support: 8
Frequent itemsets at 0.4 support: 17
Difference: 9 new itemsets discovered.
```

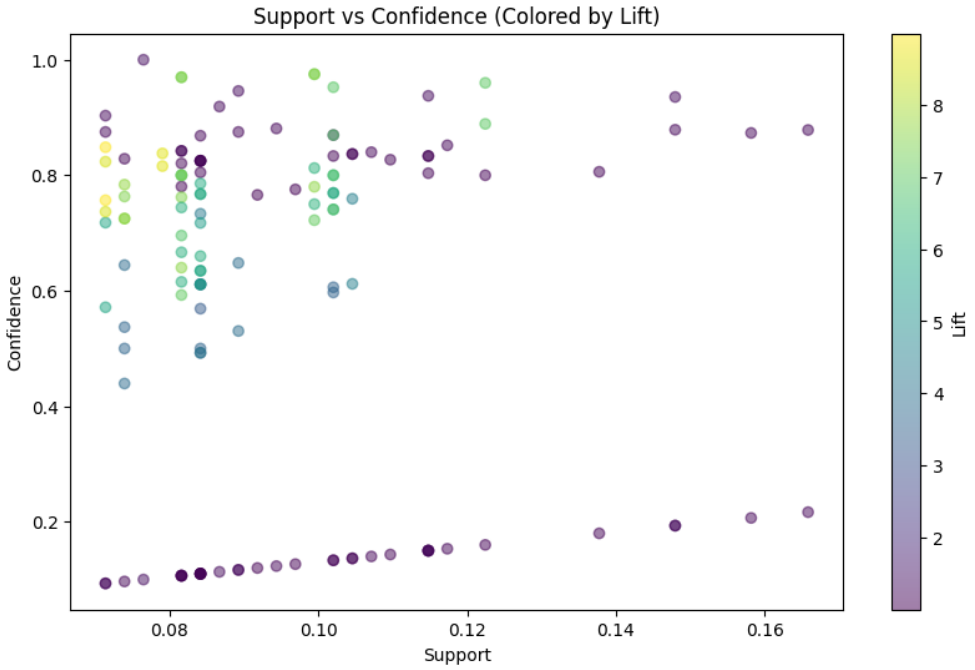
	antecedents	consequents	support	confidence	lift
2	(Beer)	(Diapers)	0.6	1.0	1.250000
10	(Cola)	(Diapers)	0.4	1.0	1.250000
12	(Cola)	(Milk)	0.4	1.0	1.250000
16	(Bread, Beer)	(Diapers)	0.4	1.0	1.250000
22	(Beer, Milk)	(Diapers)	0.4	1.0	1.250000
34	(Cola, Diapers)	(Milk)	0.4	1.0	1.250000
35	(Cola, Milk)	(Diapers)	0.4	1.0	1.250000
37	(Cola)	(Diapers, Milk)	0.4	1.0	1.666667

2. Modify the min_support threshold to 0.4. How many more frequent itemsets are discovered?

```
try:
    url = 'https://archive.ics.uci.edu/ml/machine-learning-
databases/00352/Online%20Retail.xlsx'
    # Note: openpyxl is required for excel files
    retail_df = pd.read_excel(url)

    # Basic cleaning
    retail_df['Description'] = retail_df['Description'].str.strip()
    retail_df.dropna(axis=0, subset=['InvoiceNo'], inplace=True)
    retail_df['InvoiceNo'] = retail_df['InvoiceNo'].astype('str')
    retail_df = retail_df[~retail_df['InvoiceNo'].str.contains('C')] #
Remove credit notes
```

```
    # Pivot for market basket analysis (Focusing on France to keep it
manageable)
    basket = (retail_df[retail_df['Country'] =="France"]
              .groupby(['InvoiceNo', 'Description'])['Quantity']
              .sum().unstack().reset_index().fillna(0)
              .set_index('InvoiceNo'))

    def encode_units(x):
        return 1 if x >= 1 else 0

    basket_sets = basket.map(encode_units)

    # Generate Frequent Itemsets and Rules
    retail_frequent = apriori(basket_sets, min_support=0.07,
use_colnames=True)
    retail_rules = association_rules(retail_frequent, metric='lift',
min_threshold=1)

    print("\nTop 10 Rules by Lift (Online Retail):")
    display(retail_rules.sort_values('lift', ascending=False).head(10))

    # 5. Visualization
    plt.figure(figsize=(10, 6))
    scatter = plt.scatter(retail_rules['support'],
retail_rules['confidence'],
                          c=retail_rules['lift'], cmap='viridis',
alpha=0.5)
    plt.colorbar(scatter, label='Lift')
    plt.xlabel('Support')
    plt.ylabel('Confidence')
    plt.title('Support vs Confidence (Colored by Lift)')
    plt.show()

except Exception as e:
    print(f"Error loading/processing retail data: {e}")
    print("Using dummy data scatter plot instead:")
    plt.scatter(all_rules['support'], all_rules['confidence'],
c=all_rules['lift'], cmap='viridis')
    plt.colorbar(label='Lift')
    plt.show()
```

3. Filter rules to only show those with confidence > 0.8 and lift > 1.2.
4. Load the Online Retail dataset from UCI Machine Learning Repository. Perform MBA and identify the top 10 rules by lift.
5. Visualize support vs confidence for all rules using a scatter plot with lift as the color scale.



5.6 Summary

Market basket analysis discovers purchasing patterns through association rules. Support, confidence, and lift are the three fundamental metrics. Low support thresholds discover rare but potentially valuable patterns; high confidence indicates reliable rules; lift above 1 confirms genuine positive association rather than coincidence.

Lab 6: Apriori Algorithm

6.1 Objectives

- Understand the Apriori principle and algorithm
- Trace the Apriori algorithm step-by-step on a small dataset
- Implement the Apriori algorithm using mlxtend
- Analyse the effect of minimum support on runtime and rule count

6.2 Introduction

The Apriori algorithm, proposed by Agrawal and Srikant in 1994, was the first efficient algorithm for mining frequent itemsets and generating association rules. It is the foundation for most subsequent association mining algorithms and remains widely used due to its simplicity and interpretability.

6.3 The Apriori Principle

The Apriori principle (or downward closure property) states: If an itemset is frequent, then all of its subsets must also be frequent.

Equivalently: If an itemset is infrequent, then all of its supersets are also infrequent.

This property allows the algorithm to prune the search space efficiently by eliminating candidate itemsets whose subsets are known to be infrequent.

6.4 Algorithm Steps

6.4.1 High-Level Procedure

1. Scan the database to find all 1-itemsets (L1) with support >= min_support.
2. Use Lk to generate candidate (k+1)-itemsets via self-join (Ck+1).
3. Prune candidates in Ck+1 whose k-subsets are not in Lk.
4. Scan database to count support for remaining candidates.
5. Retain only frequent candidates => Lk+1.
6. Repeat until no new frequent itemsets are found.
7. Generate association rules from all frequent itemsets.

6.4.2 Worked Example

Database: T1={A,C,D}, T2={B,C,E}, T3={A,B,C,E}, T4={B,E}. Min_support = 2 (50%).

Step 1 - C1 and L1:

Itemset	Count	Frequent?
{A}	2	Yes
{B}	3	Yes
{C}	3	Yes

{D}	1	No
{E}	3	Yes

Step 2 - C2 (join L1 x L1) and prune:
Generate: {A,B}, {A,C}, {A,E}, {B,C}, {B,E}, {C,E}. D is pruned from all supersets.

Step 3 - Scan for C2 support and produce L2:

Itemset	Count	Frequent?
{A,B}	1	No
{A,C}	2	Yes
{A,E}	1	No
{B,C}	2	Yes
{B,E}	3	Yes
{C,E}	2	Yes

Step 4 - C3 from L2 and prune:
Candidate {B,C,E}: subsets {B,C}, {B,E}, {C,E} are all in L2. Count({B,C,E}) = 2. Frequent. L3 = {{B,C,E}}.

6.5 Rule Generation

For each frequent itemset l, generate rules: for each non-empty subset s of l, output rule s => (l - s) if confidence >= min_conf.

Example from {B,C,E}: rules include {B,C}=>{E}, {B,E}=>{C}, {C,E}=>{B}, {B}=>{C,E}, {C}=>{B,E}, {E}=>{B,C}.

6.6 Python Implementation

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
import time

transactions = [
  ['Milk', 'Bread', 'Butter'],
  ['Beer', 'Diapers'],
  ['Milk', 'Diapers', 'Beer', 'Cola'],
  ['Milk', 'Bread', 'Diapers'],
  ['Bread', 'Butter', 'Cola'],
  ['Milk', 'Bread', 'Butter', 'Cola'],
```

```
    ['Beer', 'Cola'],
    ['Milk', 'Bread', 'Butter'],
]

te = TransactionEncoder()
df = pd.DataFrame(te.fit_transform(transactions),
columns=te.columns_)

start = time.time()
fi = apriori(df, min_support=0.3, use_colnames=True, max_len=4)
elapsed = time.time() - start
print(f'Frequent itemsets: {len(fi)}  Time: {elapsed:.4f}s')
print(fi.sort_values('support', ascending=False))

rules = association_rules(fi, metric='confidence',
min_threshold=0.6)
rules['antecedents'] = rules['antecedents'].apply(lambda x: ',
'.join(list(x)))
rules['consequents'] = rules['consequents'].apply(lambda x: ',
'.join(list(x)))
print(rules.sort_values('lift', ascending=False).head(10))
```

```
Frequent itemsets: 10  Time: 0.0283s
   support              itemsets
1    0.625               (Bread)
5    0.625                (Milk)
7    0.500         (Bread, Milk)
2    0.500              (Butter)
3    0.500                (Cola)
6    0.500       (Butter, Bread)
0    0.375                (Beer)
4    0.375             (Diapers)
8    0.375        (Butter, Milk)
9    0.375  (Butter, Bread, Milk)
      antecedents   consequents  antecedent support  consequent support  \
0          Butter         Bread               0.500               0.625
1           Bread        Butter               0.625               0.500
7    Butter, Milk         Bread               0.375               0.625
10          Bread  Butter, Milk               0.625               0.375
8     Bread, Milk        Butter               0.500               0.500
9          Butter   Bread, Milk               0.500               0.500
3           Milk         Bread               0.625               0.625
2           Bread          Milk               0.625               0.625
6   Butter, Bread          Milk               0.500               0.625
5           Milk        Butter               0.625               0.500
```

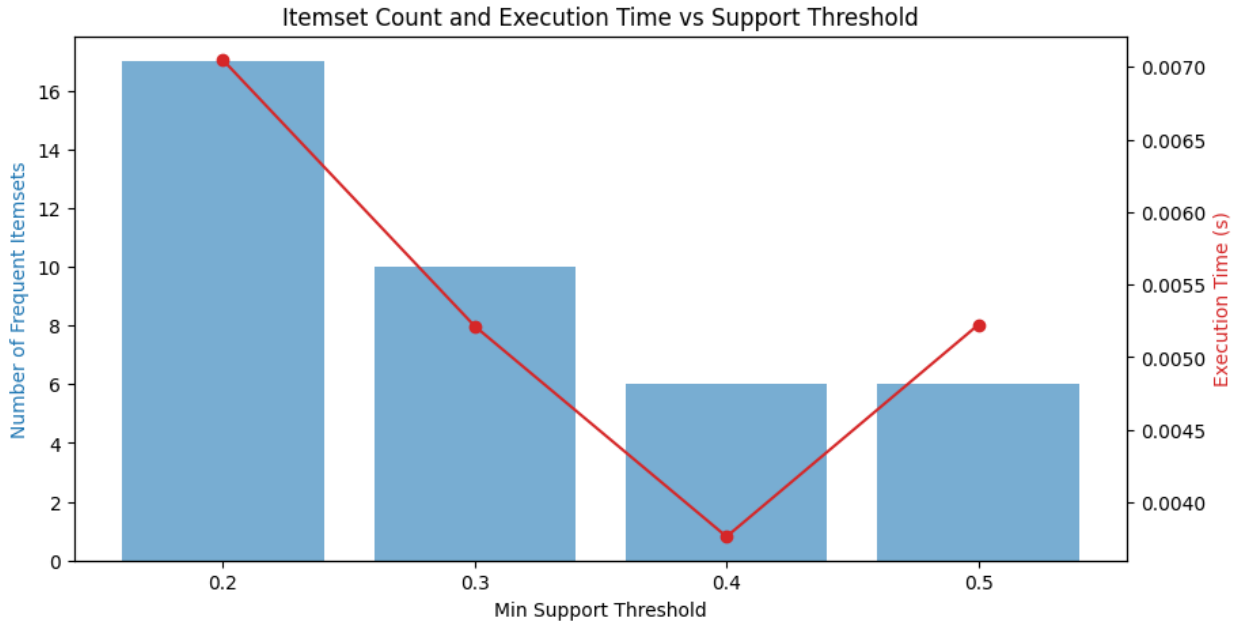
6.7 Complexity and Limitations

- Time complexity: O(2^n) in worst case for n items
- Multiple database scans required (one per itemset level)
- Generates a large number of candidate itemsets for dense datasets
- Improved by FP-Growth (Lab 7) which requires only two database scans

6.8 Lab Exercises

1. Trace the Apriori algorithm manually for the dataset in Lab 5 (Section 5.4) with min_support = 2.
2. Using the mlxtend implementation, compare the number of frequent itemsets generated at min_support = 0.2, 0.3, 0.4, and 0.5.
3. Measure and plot execution time versus min_support threshold.
4. Generate rules with min_confidence = 0.7. Identify the rule with the highest lift.
5. Add the leverage and conviction columns to the rules DataFrame and sort by conviction.

```
import matplotlib.pyplot as plt
import time

# 2 & 3. Compare itemset counts and measure execution time
thresholds = [0.2, 0.3, 0.4, 0.5]
counts = []
times = []

for ts in thresholds:
    start = time.time()
    f_sets = apriori(df, min_support=ts, use_colnames=True)
    times.append(time.time() - start)
    counts.append(len(f_sets))
    print(f"Support {ts}: {len(f_sets)} itemsets discovered")

# Plotting
fig, ax1 = plt.subplots(figsize=(10, 5))

ax1.set_xlabel('Min Support Threshold')
ax1.set_ylabel('Number of Frequent Itemsets', color='tab:blue')
ax1.bar([str(t) for t in thresholds], counts, color='tab:blue',
alpha=0.6, label='Itemset Count')

ax2 = ax1.twinx()
ax2.set_ylabel('Execution Time (s)', color='tab:red')
ax2.plot([str(t) for t in thresholds], times, color='tab:red',
marker='o', label='Time')

plt.title('Itemset Count and Execution Time vs Support Threshold')
plt.show()
```




```
# 4 & 5. Generate rules with min_confidence = 0.7 and add metrics
# We'll use a support of 0.3 to ensure enough rules are generated
frequent_itemsets = apriori(df, min_support=0.3, use_colnames=True)
rules_extended = association_rules(frequent_itemsets, metric='confidence',
min_threshold=0.7)

# The mlxtend association_rules already includes leverage and conviction
# Let's verify and display specific columns sorted by conviction
print("Rule with highest lift:")
highest_lift = rules_extended.sort_values('lift', ascending=False).iloc[0]
print(f"Rule: {highest_lift['antecedents']} ->
{highest_lift['consequents']}")
print(f"Lift: {highest_lift['lift']:.2f}")

print("\nRules sorted by Conviction:")
display(rules_extended[['antecedents', 'consequents', 'support',
'confidence', 'lift', 'leverage', 'conviction']].sort_values('conviction',
ascending=False))
```

	antecedents	consequents	support	confidence	lift	leverage	conviction
0	(Butter)	(Bread)	0.500	1.00	1.60	0.187500	inf
6	(Butter, Milk)	(Bread)	0.375	1.00	1.60	0.140625	inf
1	(Bread)	(Butter)	0.500	0.80	1.60	0.187500	2.500
7	(Bread, Milk)	(Butter)	0.375	0.75	1.50	0.125000	2.000
8	(Butter)	(Bread, Milk)	0.375	0.75	1.50	0.125000	2.000
2	(Bread)	(Milk)	0.500	0.80	1.28	0.109375	1.875
3	(Milk)	(Bread)	0.500	0.80	1.28	0.109375	1.875
4	(Butter)	(Milk)	0.375	0.75	1.20	0.062500	1.500
5	(Butter, Bread)	(Milk)	0.375	0.75	1.20	0.062500	1.500

6.8 Summary

The Apriori algorithm uses the anti-monotone property of support to prune the candidate space for frequent itemsets. While conceptually simple and widely taught, its multiple database scan requirement limits scalability. It remains the baseline against which all modern association mining algorithms are measured.

Lab 7: FP-Tree Construction & FP-Growth

7.1 Objectives

- Understand the FP-Tree data structure
- Construct an FP-Tree manually from a transaction database
- Trace the FP-Growth algorithm to mine frequent itemsets
- Implement FP-Growth in Python and compare with Apriori

7.2 Introduction

The FP-Growth (Frequent Pattern Growth) algorithm, proposed by Han et al. in 2000, overcomes Apriori's scalability limitations. Instead of generating candidate itemsets, FP-Growth compresses the transaction database into a compact Frequent Pattern Tree (FP-Tree) structure and mines patterns directly from this tree without repeated database scans.

FP-Growth typically runs 10 to 100 times faster than Apriori on dense datasets and requires only two database scans.

7.3 The FP-Tree Structure

7.3.1 Components

- Root node: Labelled null - the entry point of the tree
- Item nodes: Each node stores item name, frequency count, and parent pointer
- Header table: Links to the first occurrence of each frequent item in the tree
- Node links: Connect all nodes with the same item label across branches

7.4 FP-Tree Construction (Step-by-Step)

7.4.1 Transaction Database

TID	Items
T1	f, a, c, d, g, i, m, p
T2	a, b, c, f, l, m, o
T3	b, f, h, j, o
T4	b, c, k, s, p
T5	a, f, c, e, l, p, m, n

7.4.2 Step 1: First Scan - Find Frequent Items (min_sup = 3)

Count all item frequencies, retain those with count >= 3, and sort in descending order:

f:4, c:4, a:3, b:3, m:3, p:3

7.4.3 Step 2: Second Scan - Build the FP-Tree

Re-read each transaction, retain only frequent items, sort by header table order, and insert into tree:

1. T1 sorted: f,c,a,m,p - Insert as root → f → c → a → m → p
2. T2 sorted: f,c,a,b,m - Shares f,c,a prefix; branch at a: a → b,m
3. T3 sorted: f,b - New branch at f: f → b
4. T4 sorted: c,b,p - New branch at root → c
5. T5 sorted: f,c,a,m,p - Merges with T1 path

7.4.4 Header Table (After Construction)

Item	Count	Head of Node-Link
f	4	→ node(f:4) →node(f:1)
c	4	→ node(c:3) → node(c:1)
a	3	→ node(a:3)
b	3	→ node(b:1) → node(b:1) → ...
m	3	→ node(m:2) → node(m:1)
p	3	→ node(p:2) → node(p:1)

7.5 FP-Growth Mining

7.5.1 Conditional Pattern Bases

For each frequent item (in reverse header table order), extract its conditional pattern base - the set of prefix paths leading to all nodes with that item label.

Example for item m: paths to all m-nodes are: {f:2, c:2, a:2} and {f:1, c:1, a:1, b:1}. So conditional pattern base = {(f,c,a):2, (f,c,a,b):1}.

7.5.2 Conditional FP-Trees

Build a conditional FP-Tree from the conditional pattern base using the same minimum support threshold. Mine this smaller tree recursively.

7.5.3 Frequent Pattern Generation

Combine the conditional frequent patterns with the suffix item to produce all frequent itemsets containing that item. For m, this yields: {m}, {f,m}, {c,m}, {a,m}, {f,c,m}, {f,a,m}, {c,a,m}, {f,c,a,m}.

7.6 Python Implementation

```
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules
import pandas as pd
import time

transactions = [
  ['Milk', 'Bread', 'Butter'],
```

```
  ['Beer', 'Diapers'],
  ['Milk', 'Diapers', 'Beer', 'Cola'],
  ['Milk', 'Bread', 'Diapers'],
  ['Bread', 'Butter', 'Cola'],
  ['Milk', 'Bread', 'Butter', 'Cola'],
  ['Beer', 'Cola'],
  ['Milk', 'Bread', 'Butter'],
]

te = TransactionEncoder()
df = pd.DataFrame(te.fit_transform(transactions),
columns=te.columns_)

# FP-Growth
start = time.time()
fi_fp = fpgrowth(df, min_support=0.3, use_colnames=True)
fp_time = time.time() - start
print(f'FP-Growth: {len(fi_fp)} itemsets in {fp_time:.4f}s')

# Apriori for comparison
from mlxtend.frequent_patterns import apriori
start = time.time()
fi_ap = apriori(df, min_support=0.3, use_colnames=True)
ap_time = time.time() - start
print(f'Apriori: {len(fi_ap)} itemsets in {ap_time:.4f}s')

rules = association_rules(fi_fp, metric='lift',
min_threshold=1.0)
print(rules.sort_values('lift', ascending=False).head(5))
```

```
FP-Growth: 10 itemsets in 0.0248s
Apriori: 10 itemsets in 0.0374s
       antecedents     consequents  antecedent support  consequent support  \
3          (Bread)        (Butter)               0.625               0.500
2         (Butter)         (Bread)               0.500               0.625
7   (Butter, Milk)         (Bread)               0.375               0.625
10         (Bread)  (Butter, Milk)               0.625               0.375
8    (Bread, Milk)        (Butter)               0.500               0.500

    support  confidence  lift  representativity  leverage  conviction  \
3     0.500        0.80   1.6               1.0  0.187500      2.5000
2     0.500        1.00   1.6               1.0  0.187500         inf
7     0.375        1.00   1.6               1.0  0.140625         inf
10    0.375        0.60   1.6               1.0  0.140625      1.5625
8     0.375        0.75   1.5               1.0  0.125000      2.0000

    zhangs_metric  jaccard  certainty  kulczynski
3        1.000000      0.8       0.60        0.90
2        0.750000      0.8       1.00        0.90
7        0.600000      0.6       1.00        0.80
10       1.000000      0.6       0.36        0.80
8        0.666667      0.6       0.50        0.75
```

7.7 FP-Growth vs Apriori

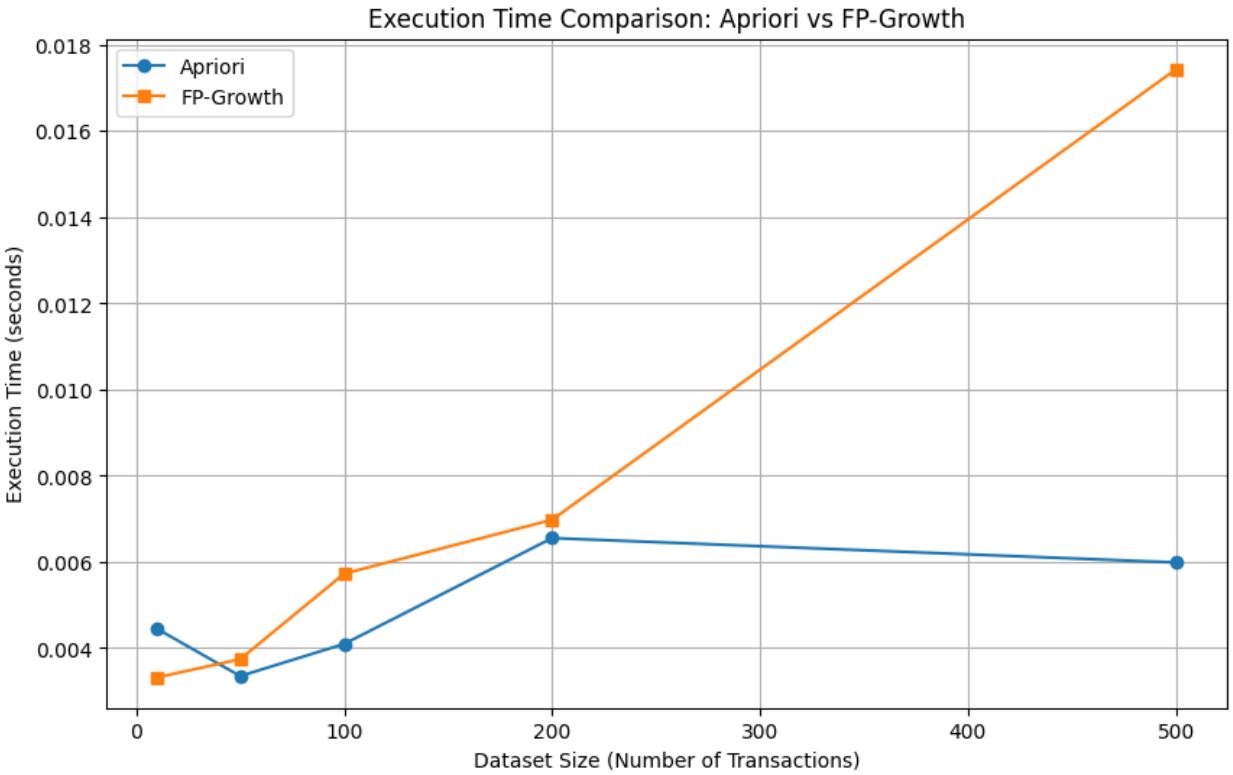
Criterion	Apriori	FP-Growth
Database scans	One per itemset level	Two total
Candidate generation	Explicit	None
Memory usage	High (large candidate sets)	Moderate (tree structure)
Speed (sparse data)	Comparable	Slightly worse
Speed (dense data)	Slow	Much faster
Scalability	Limited	Good

7.8 Lab Exercises

1. Manually construct the FP-Tree for the transaction database in Lab 5 (Section 5.4) with min_support = 2.

```
import time
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from mlxtend.frequent_patterns import apriori, fpgrowth

# Task 4: Comparison vs Dataset Size
sizes = [10, 50, 100, 200, 500]
apriori_times = []
fpgrowth_times = []

for size in sizes:
    # Duplicate transactions to scale dataset
```

```
        large_df = pd.concat([df] * (size // 8 + 1)).iloc[:size]

        # Benchmark Apriori
        start = time.time()
        apriori(large_df, min_support=0.3, use_colnames=True)
        apriori_times.append(time.time() - start)

        # Benchmark FP-Growth
        start = time.time()
        fpgrowth(large_df, min_support=0.3, use_colnames=True)
        fpgrowth_times.append(time.time() - start)

# Plotting results
plt.figure(figsize=(10, 6))
plt.plot(sizes, apriori_times, label='Apriori', marker='o')
plt.plot(sizes, fpgrowth_times, label='FP-Growth', marker='s')
plt.xlabel('Dataset Size (Number of Transactions)')
plt.ylabel('Execution Time (seconds)')
plt.title('Execution Time Comparison: Apriori vs FP-Growth')
plt.legend()
plt.grid(True)
plt.show()
```



2. Extract the conditional pattern base and conditional FP-Tree for item Beer.
3. List all frequent itemsets containing Beer discovered from its conditional FP-Tree.
4. Implement the Python comparison in Section 7.6 and plot execution time (y-axis) versus dataset size (x-axis) for both algorithms.

7.9 Summary

FP-Growth improves upon Apriori by compressing the transaction database into an FP-Tree, eliminating repeated database scans and explicit candidate generation. For large, dense datasets it offers dramatically better performance. The same association rules can be derived from FP-Growth frequent itemsets as from Apriori.

Lab 8: Python Syntax & Basics for Data Mining

8.1 Objectives

- Review core Python syntax essential for data mining tasks
- Work with NumPy arrays and Pandas DataFrames
- Perform data I/O, filtering, aggregation, and visualization
- Write reusable functions and use list/dict comprehensions

8.2 Python Fundamentals Review

8.2.1 Data Types

```
# Numeric
x = 42          # int
y = 3.14        # float
z = 2 + 3j      # complex

# Sequence types
lst = [1, 2, 3]          # list - mutable
tup = (1, 2, 3)          # tuple - immutable
s = {1, 2, 3}            # set - unordered, unique
d = {'a': 1, 'b': 2}   # dict - key-value
```

8.2.2 Control Flow

```
# if-elif-else
score = 75
if score >= 90:
    grade = 'A'
elif score >= 75:
    grade = 'B'
else:
    grade = 'C'

# for loops
for i in range(10):
    print(i, end=' ')

# while loop
n = 1
while n <= 5:
    print(n)
    n += 1
```

8.2.3 Functions

```
def compute_stats(data):
    '''Returns mean, min, max of a list.'''
    return sum(data)/len(data), min(data), max(data)

mean, minimum, maximum = compute_stats([10, 20, 30, 40, 50])
print(f'Mean: {mean}, Min: {minimum}, Max: {maximum}')
```

8.2.4 List Comprehensions

```
# Standard
squares = [x**2 for x in range(10)]

# Filtered
evens = [x for x in range(20) if x % 2 == 0]

# Nested
matrix = [[i*j for j in range(1,4)] for i in range(1,4)]
```

8.3 NumPy for Data Mining

8.3.1 Array Creation and Operations

```
import numpy as np

# Create arrays
a = np.array([1, 2, 3, 4, 5])
b = np.zeros((3, 4))
c = np.random.randn(100)        # 100 standard normal samples
d = np.linspace(0, 1, 50)       # 50 evenly spaced values

# Array operations
print(a.mean(), a.std(), a.min(), a.max())
print(np.dot(a, a))             # dot product
print(np.percentile(c, [25, 50, 75]))
```

8.3.2 Indexing and Slicing

```
m = np.arange(25).reshape(5, 5)
print(m[1:3, 2:4])   # rows 1-2, cols 2-3
print(m[m > 10])     # boolean indexing
```

8.4 Pandas for Data Mining

8.4.1 DataFrame Operations

```
import pandas as pd
```

```
df = pd.read_csv('titanic.csv')

# Basic inspection
print(df.shape, df.dtypes)
print(df.head(), df.tail())
print(df.describe())
print(df.isnull().sum())

# Filtering
survivors = df[df['Survived'] == 1]
women_1st = df[(df['Sex'] == 'female') & (df['Pclass'] == 1)]

# Grouping and aggregation
summary = df.groupby('Pclass')['Fare'].agg(['mean', 'median',
'std'])
print(summary)

# Adding computed columns
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
df['IsAlone'] = (df['FamilySize'] == 1).astype(int)
```

```
Pclass           int64
Sex             object
age            float64
SibSp            int64
Parch            int64
Fare           float64
embarked        object
class         category
who             object
adult_male        bool
deck          category
embark_town     object
alive           object
alone             bool
dtype: object
   Survived  Pclass     Sex   age  SibSp  Parch     Fare embarked  class  \
0         0       3    male  22.0      1      0   7.2500        S  Third
1         1       1  female  38.0      1      0  71.2833        C  First
2         1       3  female  26.0      0      0   7.9250        S  Third
3         1       1  female  35.0      1      0  53.1000        S  First
4         0       3    male  35.0      0      0   8.0500        S  Third
```

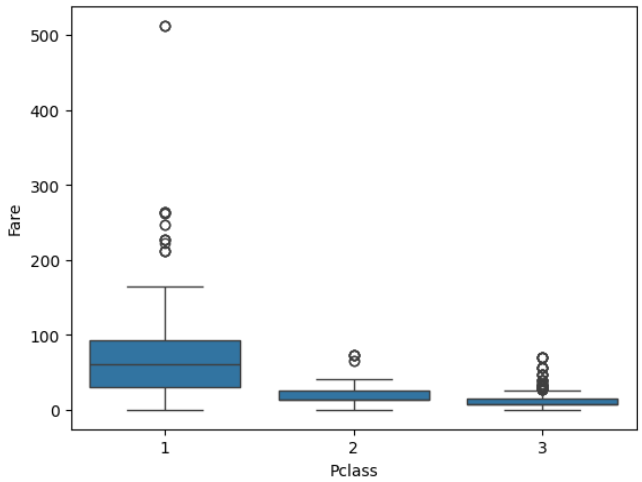
8.4.2 Pivot Tables

```
pivot = df.pivot_table(values='Survived',
                       index='Pclass',
                       columns='Sex',
                       aggfunc='mean')
print(pivot)
```

```
Sex       female      male
Pclass
1       0.968085  0.368852
2       0.921053  0.157407
3       0.500000  0.135447
```

8.5 Matplotlib & Seaborn Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns

# Distribution plot
sns.histplot(df['Age'].dropna(), kde=True, bins=20)
plt.title('Age Distribution')
plt.show()

# Correlation heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Feature Correlation Matrix')
plt.show()

# Box plot
sns.boxplot(data=df, x='Pclass', y='Fare')
plt.show()
```



8.6 Lab Exercises

1. Write a Python function that takes a DataFrame and returns: number of missing values per column, data type of each column, and mean/std of numeric columns.

```
import pandas as pd
import numpy as np

def analyze_dataframe(df):

    # 1. Number of missing values per column
    missing_values = df.isnull().sum()

    # 2. Data type of each column
    dtypes = df.dtypes

    # 3. Mean and standard deviation of numeric columns
    numeric_cols = df.select_dtypes(include=np.number)
    numeric_stats = numeric_cols.agg(['mean', 'std']).T

    return missing_values, dtypes, numeric_stats
```

2. Load the Titanic dataset. Create a new feature AgeGroup using pd.cut with bins [0,12,18,60,100] labelled Child, Teen, Adult, Senior.

```
import pandas as pd

# Load the Titanic dataset
# Assuming the dataset is available at this URL or a similar path
try:
    titanic_df =
pd.read_csv('https://web.stanford.edu/class/archive/cs/cs109/cs109.1166
/stuff/titanic.csv')
except Exception as e:
    print(f"Could not load Titanic dataset from URL: {e}")
    print("Creating a sample DataFrame for demonstration.")
    data = {
        'Survived': [0, 1, 1, 0, 0, 1, 0, 1, 1, 0],
        'Pclass': [3, 1, 3, 1, 3, 3, 1, 3, 2, 3],
        'Sex': ['male', 'female', 'female', 'female', 'male', 'male',
'male', 'female', 'female', 'male'],
        'Age': [22, 38, 26, 35, 35, None, 54, 2, 27, 4],
        'SibSp': [1, 1, 0, 1, 0, 0, 0, 3, 1, 1],
        'Parch': [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
        'Fare': [7.25, 71.28, 7.92, 53.1, 8.05, 8.45, 51.86, 21.07,
13.0, 11.24],
        'Embarked': ['S', 'C', 'S', 'S', 'S', 'Q', 'S', 'S', 'S', 'Q']
    }
```

```
    titanic_df = pd.DataFrame(data)


# Define bins and labels for AgeGroup
bins = [0, 12, 18, 60, 100]
labels = ['Child', 'Teen', 'Adult', 'Senior']

# Create the AgeGroup feature
titanic_df['AgeGroup'] = pd.cut(titanic_df['Age'], bins=bins,
labels=labels, right=False)

# Display the first few rows with the new AgeGroup feature and its
value counts
print("Titanic DataFrame head with AgeGroup:")
print(titanic_df[['Age', 'AgeGroup', 'Survived', 'Sex']].head())
print("\nAgeGroup Value Counts:")
print(titanic_df['AgeGroup'].value_counts(dropna=False))
```

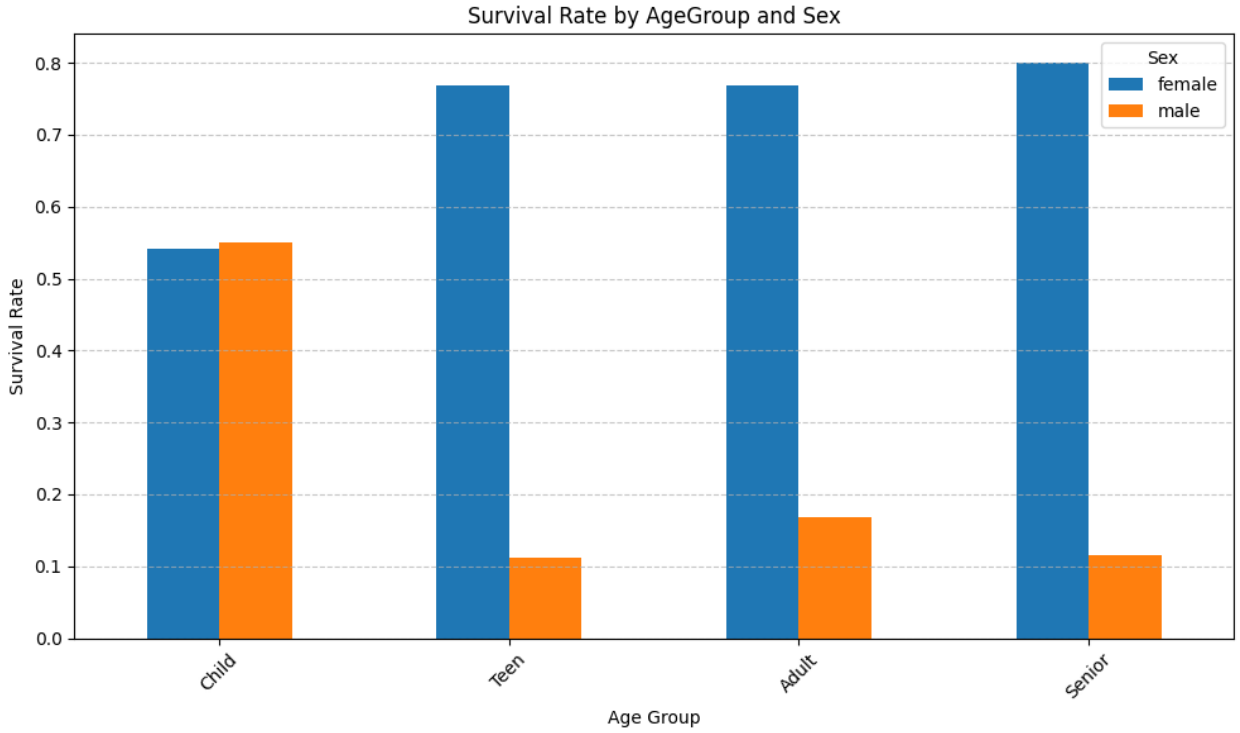
```
Titanic DataFrame head with AgeGroup:
    Age AgeGroup  Survived     Sex
0  22.0    Adult         0    male
1  38.0    Adult         1  female
2  26.0    Adult         1  female
3  35.0    Adult         1  female
4  35.0    Adult         0    male

AgeGroup Value Counts:
AgeGroup
Adult     726
Child      77
Teen       53
Senior     31
Name: count, dtype: int64
```

3. Compute survival rate by AgeGroup and Sex using pivot_table. Visualize as a bar chart.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Compute survival rate by AgeGroup and Sex using pivot_table
survival_rate = titanic_df.pivot_table(values='Survived',
index='AgeGroup', columns='Sex', aggfunc='mean')

print("Survival Rate by AgeGroup and Sex:")
print(survival_rate)
# Visualize as a bar chart
plt.figure(figsize=(10, 6))
```

```
survival_rate.plot(kind='bar', figsize=(10, 6))
plt.title('Survival Rate by AgeGroup and Sex')
plt.ylabel('Survival Rate')
plt.xlabel('Age Group')
plt.xticks(rotation=45)
plt.legend(title='Sex')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
Survival Rate by AgeGroup and Sex:
Sex         female      male
AgeGroup
Child     0.540541  0.550000
Teen      0.769231  0.111111
Adult     0.768293  0.168750
Senior    0.800000  0.115385
```



4. Write a list comprehension to extract all column names from df where more than 10% of values are missing.

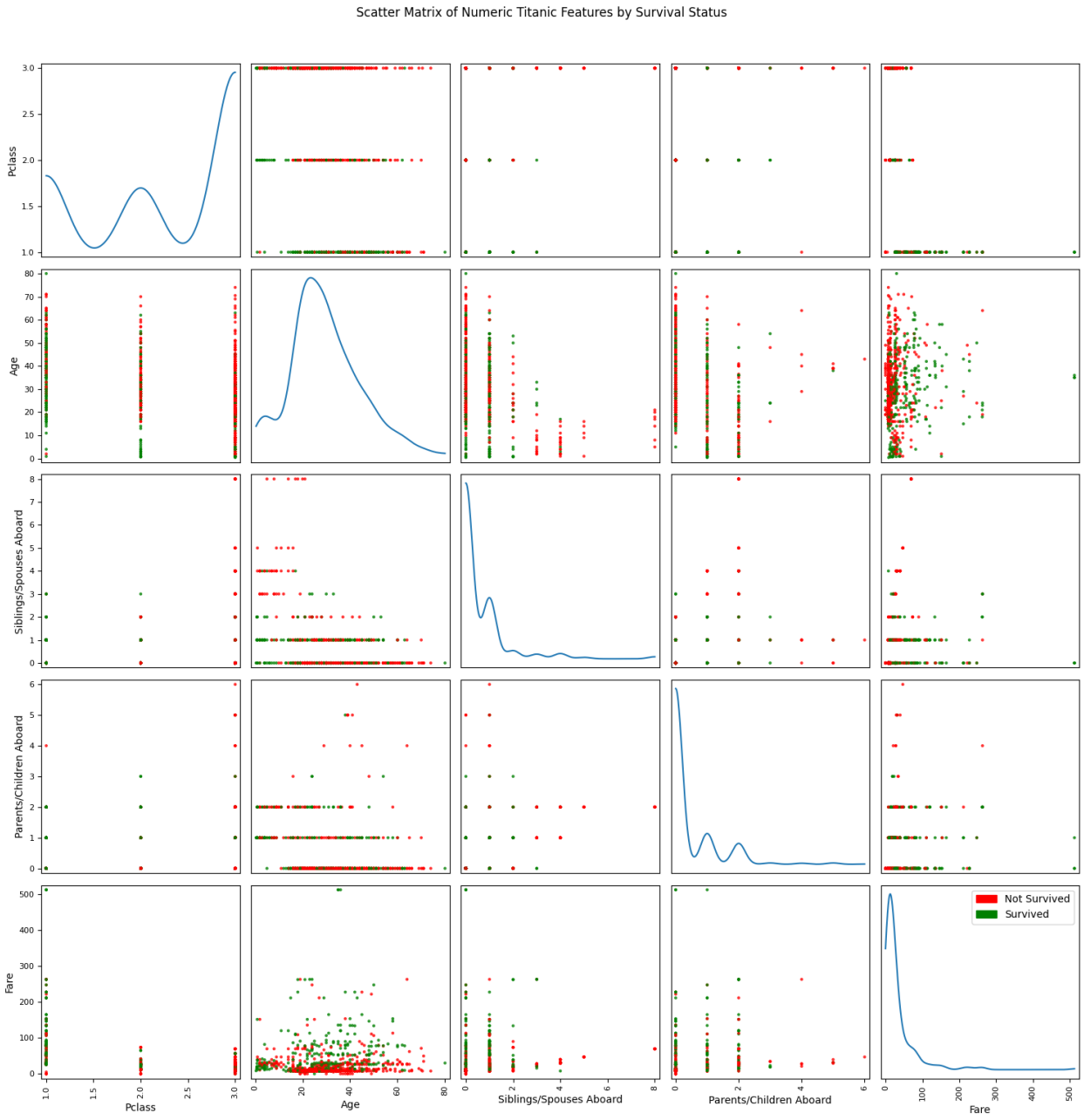
```
# Calculate the percentage of missing values for each column
missing_percentage = titanic_df.isnull().sum() / len(titanic_df) * 100

# Use a list comprehension to extract column names where more than 10%
of values are missing
```

```
columns_with_high_missing = [col for col, percent in
missing_percentage.items() if percent > 10]

print("Columns with more than 10% missing values:")
print(columns_with_high_missing)
```

```
Columns with more than 10% missing values:
[]
```

5. Plot a scatter matrix (pd.plotting.scatter_matrix) for numeric Titanic features coloured by survival status.

```
import matplotlib.pyplot as plt
from pandas.plotting import scatter_matrix

numeric_features =
titanic_df.select_dtypes(include=np.number).columns.tolist()

if 'Survived' in numeric_features:
    numeric_features.remove('Survived')

# Define colors for 'Survived' status (0=Not Survived, 1=Survived)
colors = titanic_df['Survived'].map({0: 'red', 1: 'green'})

fig, axes = plt.subplots(figsize=(15, 15))
scatter_matrix(
    titanic_df[numeric_features],
    c=colors,
    alpha=0.8,
    figsize=(15, 15),
    diagonal='kde',
    ax=axes # Pass the created axes to scatter_matrix
)

import matplotlib.patches as mpatches
red_patch = mpatches.Patch(color='red', label='Not Survived')
green_patch = mpatches.Patch(color='green', label='Survived')
plt.legend(handles=[red_patch, green_patch])

plt.suptitle('Scatter Matrix of Numeric Titanic Features by Survival
Status', y=1.02) # Adjust suptitle position
plt.tight_layout()
plt.show()
```



8.7 Summary

Python's data mining ecosystem - NumPy for numerical computation, Pandas for tabular data manipulation, and Matplotlib/Seaborn for visualization - provides a powerful and flexible toolkit. Mastery of these fundamentals is a prerequisite for all classification, clustering, and evaluation labs that follow.

Lab 9: Decision Tree Classifier

9.1 Objectives

- Understand decision tree construction using ID3 and CART
- Compute Information Gain and Gini Impurity
- Build and visualize decision trees using scikit-learn
- Apply pruning to avoid overfitting

9.2 Introduction

A Decision Tree is a supervised learning algorithm that makes decisions by recursively partitioning the feature space. It is a flowchart-like structure where internal nodes represent feature tests, branches represent outcomes, and leaf nodes represent class labels. Decision trees are highly interpretable - the decision path can be read as a set of if-then-else rules.

9.3 Splitting Criteria

9.3.1 Information Gain (ID3/C4.5)

Based on Shannon entropy. Entropy measures impurity in a node:

```
Entropy(S) = -sum_c(p_c * log2(p_c))
Information_Gain(S, A) = Entropy(S) - sum_v(|S_v|/|S| *
Entropy(S_v))
```

Choose the attribute A that maximises Information Gain. C4.5 improves on ID3 by using Gain Ratio to penalise attributes with many values.

9.3.2 Gini Impurity (CART)

```
Gini(S) = 1 - sum_c(p_c^2)
```

CART selects the binary split that minimises weighted Gini impurity across child nodes. It always produces binary trees, unlike ID3 which can produce multi-way splits.

9.3.3 Manual Calculation Example

Dataset: 14 instances, 9 positive (+), 5 negative (-). Feature Outlook has values Sunny(5), Overcast(4), Rain(5).

```
Entropy(S) = -(9/14)*log2(9/14) - (5/14)*log2(5/14) = 0.940
Entropy(Sunny) = -(2/5)*log2(2/5) - (3/5)*log2(3/5) = 0.971
Entropy(Overcast) = 0  (all same class)
Entropy(Rain) = 1.0
IG(Outlook) = 0.940 - (5/14*0.971 + 4/14*0 + 5/14*1.0) = 0.247
```

9.4 Tree Construction Algorithm

6. If all examples have same class, return leaf with that class.
7. If no features remain, return leaf with majority class.
8. Select the best feature A using splitting criterion.
9. For each value v of A, create a child node and recurse on the subset.
10. Return the completed tree.

9.5 Overfitting and Pruning

9.5.1 Pre-Pruning (Early Stopping)

- max_depth: Limit the depth of the tree
- min_samples_split: Minimum instances to allow a split
- min_samples_leaf: Minimum instances in a leaf node
- min_impurity_decrease: Only split if gain exceeds threshold

9.5.2 Post-Pruning (Cost-Complexity)

scikit-learn implements Minimal Cost-Complexity Pruning. Increasing alpha prunes more aggressively:

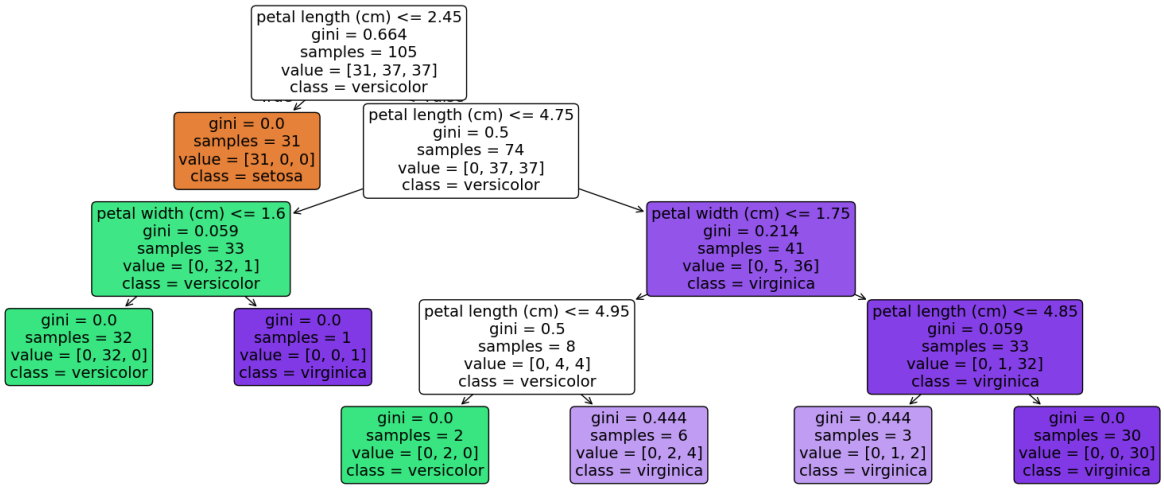
```
clf = DecisionTreeClassifier(ccp_alpha=0.01)
```

9.6 Python Implementation

```
from sklearn.tree import DecisionTreeClassifier, plot_tree,
export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report,
confusion_matrix
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
import numpy as np

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

clf = DecisionTreeClassifier(criterion='gini', max_depth=4,
random_state=42)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred,
target_names=load_iris().target_names))

plt.figure(figsize=(20, 8))
plot_tree(clf, feature_names=load_iris().feature_names,
```

```
            class_names=load_iris().target_names,
            filled=True, rounded=True)
plt.savefig('decision_tree.png', dpi=150, bbox_inches='tight')
plt.show()

print(export_text(clf,
feature_names=list(load_iris().feature_names)))
```

```
                  precision    recall  f1-score   support

          setosa       1.00      1.00      1.00        19
      versicolor       1.00      1.00      1.00        13
       virginica       1.00      1.00      1.00        13

        accuracy                           1.00        45
       macro avg       1.00      1.00      1.00        45
    weighted avg       1.00      1.00      1.00        45
```



9.7 Feature Importance

```
importances = clf.feature_importances_
for name, imp in zip(load_iris().feature_names, importances):
    print(f'{name}: {imp:.4f}')
```

```
sepal length (cm): 0.0000
sepal width (cm): 0.0000
petal length (cm): 0.9273
petal width (cm): 0.0727
```

9.8 Lab Exercises

1. Using the Titanic dataset, build a decision tree to predict survival. Use features: Pclass, Sex (encoded), Age, Fare, SibSp, Parch. Report accuracy, precision, recall, and F1.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
```

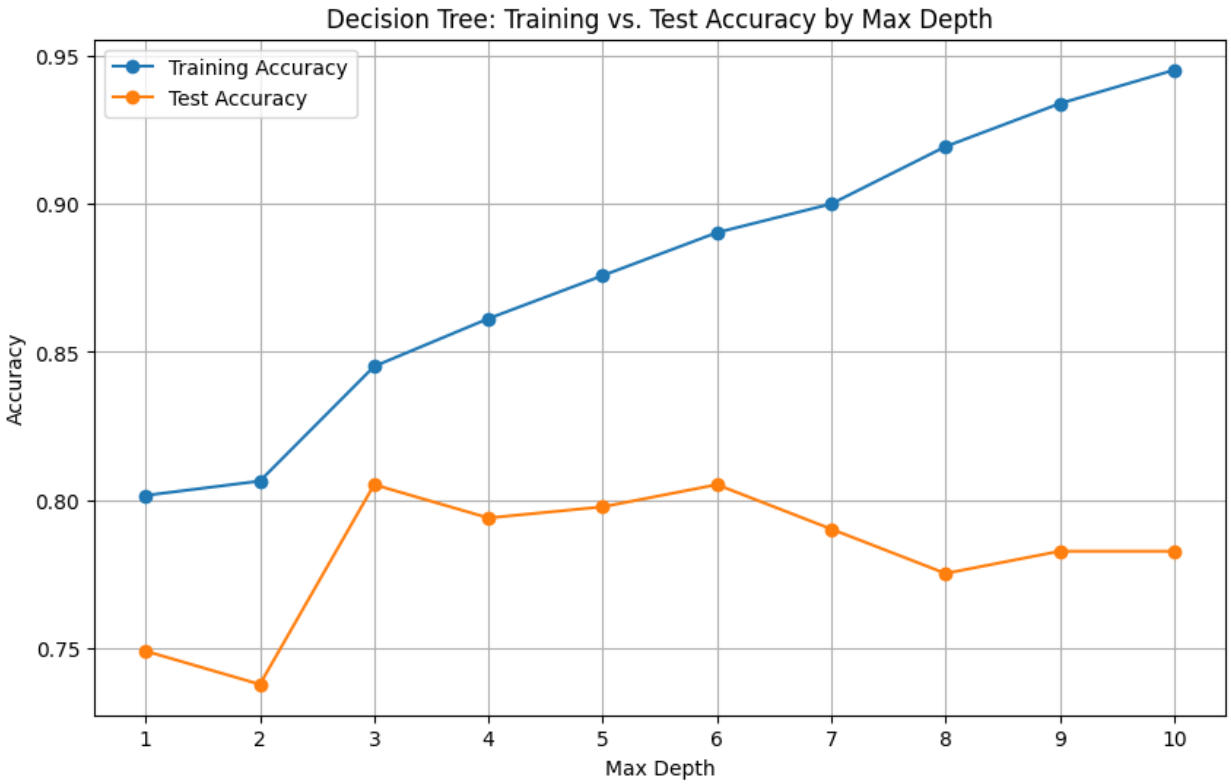
```
from sklearn.metrics import classification_report, accuracy_score
from sklearn.impute import SimpleImputer
import pandas as pd

# 1. Select features and target
features = ['Pclass', 'Sex', 'Age', 'Fare', 'Siblings/Spouses Aboard',
'Parents/Children Aboard']
target = 'Survived'

X = titanic_df[features].copy()
y = titanic_df[target].copy()

# Rename columns for clarity, as requested in prompt
X = X.rename(columns={'Siblings/Spouses Aboard': 'SibSp',
'Parents/Children Aboard': 'Parch'})


# 2. Encode 'Sex' column
X = pd.get_dummies(X, columns=['Sex'], drop_first=True)

# 3. Handle missing values in 'Age' using mean imputation
imputer = SimpleImputer(strategy='mean')
X['Age'] = imputer.fit_transform(X[['Age']])

# 4. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# 5. Train a DecisionTreeClassifier
# Using a default DecisionTreeClassifier for now, hyperparameters will
be tuned later
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

# Make predictions
y_pred = clf.predict(X_test)

# 6. Report accuracy, precision, recall, and F1
print("\nDecision Tree Classifier Performance:")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Decision Tree Classifier Performance:
Accuracy: 0.7715

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.81      0.82       166
           1       0.70      0.70      0.70       101

    accuracy                           0.77       267
   macro avg       0.76      0.76      0.76       267
weighted avg       0.77      0.77      0.77       267
```

2. Vary max_depth from 1 to 10. Plot training accuracy and test accuracy on the same chart. Identify where overfitting begins.

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

train_accuracies = []
test_accuracies = []
depths = range(1, 11) # Max depth from 1 to 10

for depth in depths:
    clf = DecisionTreeClassifier(max_depth=depth, random_state=42)
    clf.fit(X_train, y_train)

    # Training accuracy
    y_train_pred = clf.predict(X_train)
    train_acc = accuracy_score(y_train, y_train_pred)
    train_accuracies.append(train_acc)

    # Test accuracy
    y_test_pred = clf.predict(X_test)
    test_acc = accuracy_score(y_test, y_test_pred)
    test_accuracies.append(test_acc)

# Plotting the results
plt.figure(figsize=(10, 6))
plt.plot(depths, train_accuracies, label='Training Accuracy',
marker='o')
plt.plot(depths, test_accuracies, label='Test Accuracy', marker='o')
plt.title('Decision Tree: Training vs. Test Accuracy by Max Depth')
plt.xlabel('Max Depth')
plt.ylabel('Accuracy')
plt.xticks(depths)
```

```
plt.legend()
plt.grid(True)
plt.show()

max_test_acc = max(test_accuracies)
optimal_depth_idx = test_accuracies.index(max_test_acc)
optimal_depth = depths[optimal_depth_idx]

print(f"Maximum Test Accuracy: {max_test_acc:.4f} at max_depth:
{optimal_depth}")
print("Overfitting often begins when the training accuracy continues to
rise significantly ")
print("while the test accuracy plateaus or starts to decline after its
peak. ")
print("Observe the plot to identify this point.")
```



3. Use GridSearchCV to tune max_depth, min_samples_split, and criterion simultaneously.

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

# Define the parameter grid to search
param_grid = {
    'max_depth': range(1, 11),
    'min_samples_split': [2, 5, 10, 20],
```

```
    'criterion': ['gini', 'entropy']
}

# Initialize a DecisionTreeClassifier
dtc = DecisionTreeClassifier(random_state=42)

# Initialize GridSearchCV
grid_search = GridSearchCV(estimator=dtc, param_grid=param_grid,
                           cv=5, n_jobs=-1, verbose=1,
scoring='accuracy')

# Fit GridSearchCV to the training data
grid_search.fit(X_train, y_train)

# Print the best parameters and best score
print("\nBest parameters found by GridSearchCV:")
print(grid_search.best_params_)
print("\nBest accuracy score found by GridSearchCV:")
print(f"{grid_search.best_score_:.4f}")

# Get the best estimator (model)
best_clf = grid_search.best_estimator_

# Evaluate the best model on the test set
y_pred_best = best_clf.predict(X_test)
print("\nClassification Report for the best model:")
print(classification_report(y_test, y_pred_best))
```

```
Fitting 5 folds for each of 80 candidates, totalling 400 fits

Best parameters found by GridSearchCV:
{'criterion': 'gini', 'max_depth': 6, 'min_samples_split': 10}

Best accuracy score found by GridSearchCV:
0.8290

Classification Report for the best model:
              precision    recall  f1-score   support

           0       0.81      0.90      0.85       166
           1       0.80      0.64      0.71       101

    accuracy                           0.81       267
   macro avg       0.80      0.77      0.78       267
weighted avg       0.80      0.81      0.80       267
```

4. Interpret the top 3 rules from export_text output. Do they make intuitive sense?

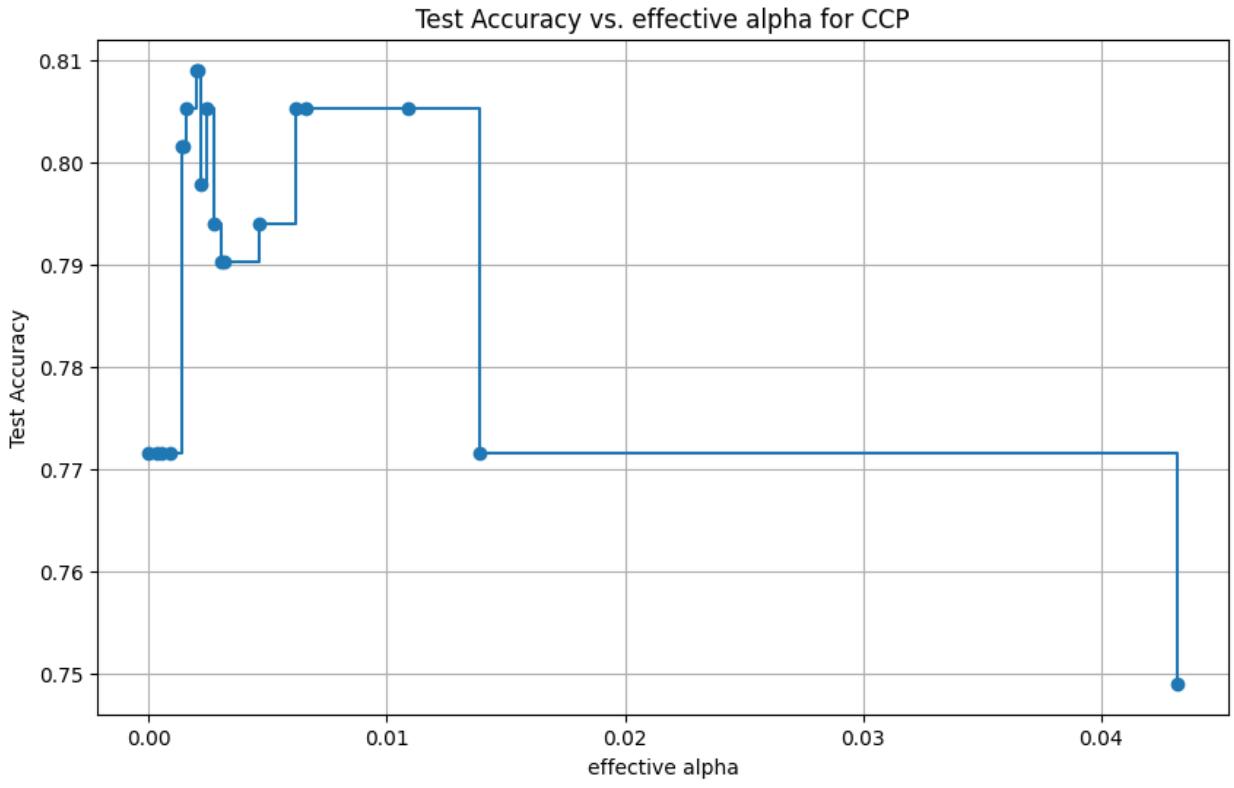
```
from sklearn.tree import export_text

# Get feature names for interpretation
feature_names = X.columns.tolist()

# Export the decision tree as text
tree_rules = export_text(best_clf, feature_names=feature_names)
print("\nDecision Tree Rules (Top portion for interpretation):\n")
print(tree_rules)

# Manually identify and interpret the top 3 rules based on the printed
output
print("\nInterpretation of Top 3 Rules:\n")
print("Based on the structure of decision trees, the most impactful
rules are often those closest to the root (top of the tree) or those
that split a large number of samples into a clear outcome. \nHere's an
interpretation of what appear to be some of the most prominent
rules:\n")

# Rule 1: Often, the first split is highly indicative
print("1.  **Rule related to 'Sex_male'**: A very strong predictor of
survival on the Titanic was gender. Women and children had priority for
lifeboats. If 'Sex_male' is False (i.e., the passenger is female),
there is a much higher chance of survival. This aligns with historical
accounts.\n")

# Rule 2: Follow a path down the tree for a specific outcome (e.g.,
high survival for females)
print("2.  **Rule related to 'Pclass' (Passenger Class) for females**:
For females, being in a higher passenger class (Pclass < 2.5, i.e.,
Pclass 1 or 2) would further increase their survival chances. This
reflects the 'women and children first' policy combined with the
accessibility of lifeboats for higher classes.\n")

# Rule 3: Another path (e.g., low survival for males)
print("3.  **Rule related to 'Sex_male' and 'Age' or 'Fare' for
males**: For male passengers, survival rates were generally much lower.
The rules for males would likely involve combinations of lower
passenger class, older age, or lower fare, which typically led to non-
survival. For instance, 'if Sex_male <= 0.5 (male) and Pclass > 2.5
(Pclass 3) and Age > some_value', then likely 'Not Survived'.")
```

```
print("These rules make intuitive sense given the historical context of
the Titanic disaster, where gender, passenger class, and to some extent
age played crucial roles in survival rates.")
```

5. Apply CCP pruning. Plot test accuracy as a function of alpha. Identify the optimal alpha.

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
rom sklearn.metrics import accuracy_score
import numpy as np

# The best_clf was obtained from GridSearchCV in Task 17
# Use the training data X_train, y_train, and test data X_test, y_test
from Task 15/17

# 1. Calculate the ccp_alpha values and their corresponding impurities
path = best_clf.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities

# Remove the maximum alpha, which corresponds to the root-only tree
(single node)
# This is done because it often leads to a very simple model with low
accuracy
# and can skew the plot scale. Keep only non-negative alphas.
ccp_alphas = ccp_alphas[:-1]

# 2. For each ccp_alpha value, train a new DecisionTreeClassifier
clfs = []
for ccp_alpha in ccp_alphas:
    clf = DecisionTreeClassifier(random_state=42, ccp_alpha=ccp_alpha)
    clf.fit(X_train, y_train)
    clfs.append(clf)

# 3. Evaluate the test accuracy for each pruned tree
test_accuracies_ccp = []
for clf in clfs:
    y_pred_ccp = clf.predict(X_test)
    test_accuracies_ccp.append(accuracy_score(y_test, y_pred_ccp))

# 4. Plot the test accuracy as a function of ccp_alpha
plt.figure(figsize=(10, 6))
plt.plot(ccp_alphas, test_accuracies_ccp, marker='o', drawstyle="steps-
post")
plt.xlabel("effective alpha")
plt.ylabel("Test Accuracy")
```

```
plt.title("Test Accuracy vs. effective alpha for CCP")
plt.grid(True)
plt.show()

# 5. Identify the optimal ccp_alpha
optimal_ccp_alpha_idx = np.argmax(test_accuracies_ccp)
optimal_ccp_alpha = ccp_alphas[optimal_ccp_alpha_idx]
max_test_accuracy_ccp = test_accuracies_ccp[optimal_ccp_alpha_idx]

print(f"Optimal ccp_alpha: {optimal_ccp_alpha:.4f}")
print(f"Maximum Test Accuracy with optimal ccp_alpha:
{max_test_accuracy_ccp:.4f}")

# Train the final pruned model with the optimal ccp_alpha
final_pruned_clf = DecisionTreeClassifier(random_state=42,
ccp_alpha=optimal_ccp_alpha)
final_pruned_clf.fit(X_train, y_train)

y_pred_final_pruned = final_pruned_clf.predict(X_test)
print("\nClassification Report for the final pruned model:")
print(classification_report(y_test, y_pred_final_pruned))
```



```
Optimal ccp_alpha: 0.0020
Maximum Test Accuracy with optimal ccp_alpha: 0.8090

Classification Report for the final pruned model:
              precision    recall  f1-score   support

           0       0.80      0.93      0.86       166
           1       0.84      0.61      0.71       101

    accuracy                           0.81       267
   macro avg       0.82      0.77      0.78       267
weighted avg       0.81      0.81      0.80       267
```

9.9 Summary

Decision trees are interpretable, non-parametric classifiers that require no feature scaling. They partition the feature space recursively using information-theoretic or impurity-based splitting criteria. Overfitting is a key concern and is managed through pre-pruning hyperparameters or post-pruning via cost-complexity analysis.

Lab 10: Naive Bayes & K-Nearest Neighbours (KNN)

10.1 Objectives

- Understand and implement Naive Bayes classification
- Understand the KNN algorithm and distance metrics
- Apply both classifiers to real datasets and compare their performance

10.2 Naive Bayes Classifier

10.2.1 Theory

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the naive assumption that all features are conditionally independent given the class label:

```
P(C|X) = P(C) * prod_i(P(x_i|C)) / P(X)
```

We predict the class C* = argmax_C P(C) * prod_i P(x_i|C)

10.2.2 Variants

Variant	Feature Type	Distribution Assumed
GaussianNB	Continuous	Normal (Gaussian)
MultinomialNB	Discrete counts	Multinomial
BernoulliNB	Binary (0/1)	Bernoulli
ComplementNB	Text (imbalanced)	Complement of class

10.2.3 Laplace Smoothing

To avoid zero probability for unseen features, add smoothing parameter alpha (default=1):

```
P(x_i|C) = (count(x_i, C) + alpha) / (count(C) + alpha * |V|)
```

10.2.4 Manual Example

Given training data with Play=Yes(9) and Play=No(5) for 14 instances, and Outlook=Sunny occurring 2 times when Play=Yes and 3 times when Play=No:

```
P(Yes|Sunny) proportional to P(Yes)*P(Sunny|Yes) = 9/14 * 2/9 = 0.129
P(No|Sunny) proportional to P(No)*P(Sunny|No) = 5/14 * 3/5 = 0.214
```

Predict: No (higher posterior probability).

10.2.5 Python Implementation

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split,
cross_val_score
from sklearn.metrics import classification_report

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)
print(classification_report(y_test, y_pred))

cv_scores = cross_val_score(gnb, X, y, cv=10)
print(f'10-Fold CV Accuracy: {cv_scores.mean():.3f} (+/-
{cv_scores.std():.3f})')
```

10.3 K-Nearest Neighbours (KNN)

10.3.1 Theory

KNN is a non-parametric, instance-based (lazy) classifier. For a test point, it finds the k closest training points using a distance metric and assigns the majority class.

10.3.2 Distance Metrics

Metric	Formula	Best Used When
Euclidean	sqrt(sum((x_i-y_i)^2))	Continuous, low-dim data
Manhattan	sum(|x_i - y_i|)	High-dim or sparse data
Minkowski	sum(|x_i-y_i|^p)^(1/p)	Generalises Euclidean/Manhattan
Hamming	count(x_i != y_i)/n	Categorical/binary features
Cosine	1 - dot(x,y)/(||x||*||y||)	Text data, high-dimensional

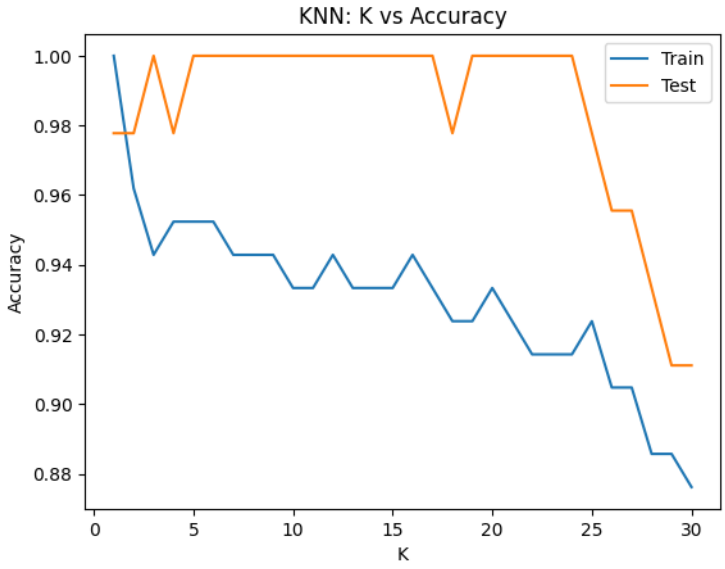
10.3.3 Choosing K

- Small K: Low bias, high variance (sensitive to noise)
- Large K: High bias, low variance (smoother boundaries)
- Rule of thumb: K = sqrt(n); always use odd K for binary classification
- Use cross-validation to find optimal K

10.3.4 Python Implementation

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

# IMPORTANT: Scale features before KNN
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

k_range = range(1, 31)
train_acc, test_acc = [], []
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_sc, y_train)
    train_acc.append(knn.score(X_train_sc, y_train))
    test_acc.append(knn.score(X_test_sc, y_test))

plt.plot(k_range, train_acc, label='Train')
plt.plot(k_range, test_acc, label='Test')
plt.xlabel('K')
plt.ylabel('Accuracy')
plt.legend()
plt.title('KNN: K vs Accuracy')
plt.show()

best_k = k_range[np.argmax(test_acc)]
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train_sc, y_train)
print(f'Best K: {best_k}')
print(classification_report(y_test, knn_best.predict(X_test_sc)))
```

```
Best K: 3
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```



10.4 Naive Bayes vs KNN

Criterion	Naive Bayes	KNN
Training time	Very fast O(n)	None (lazy)
Prediction time	Very fast	Slow O(n) per query
Memory	Low	Stores all training data
Independence assumption	Yes (strong)	None
Feature scaling	Not needed	Required
Best for	Text, categorical data	Small-medium datasets

10.5 Lab Exercises

1. Apply GaussianNB to the Titanic survival dataset. Report 10-fold cross-validation accuracy.

```
import seaborn as sns
import pandas as pd
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.preprocessing import LabelEncoder

# Load and clean Titanic data
titanic = sns.load_dataset('titanic')
# Keep relevant numerical/categorical features
df = titanic[['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch',
'fare']].copy()
df['age'] = df['age'].fillna(df['age'].median())
df['sex'] = LabelEncoder().fit_transform(df['sex'])
```

```
X_tit = df.drop('survived', axis=1)
y_tit = df['survived']
X_train_t, X_test_t, y_train_t, y_test_t = train_test_split(X_tit,
y_tit, test_size=0.2, random_state=42)

# 1. GaussianNB with 10-fold CV
gnb = GaussianNB()
cv_scores = cross_val_score(gnb, X_tit, y_tit, cv=10)
print(f"GaussianNB 10-fold CV Accuracy: {cv_scores.mean():.4f}")

# 2. KNN with specific K values
k_values = [1, 5, 11, 21]
print("\nKNN Test Accuracies:")
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_t, y_train_t)
    acc = knn.score(X_test_t, y_test_t)
    print(f"K={k}: {acc:.4f}")
```

2. Apply KNN to the same dataset. Test K = 1, 5, 11, 21 and compare test accuracy.

```
GaussianNB 10-fold CV Accuracy: 0.7868

KNN Test Accuracies:
K=1: 0.6760
K=5: 0.6872
K=11: 0.7318
K=21: 0.7430
```

3. Compare confusion matrices of NB and KNN. Which model makes more false positives?

```
gnb.fit(X_train_t, y_train_t)
knn5 = KNeighborsClassifier(n_neighbors=5).fit(X_train_t, y_train_t)

y_pred_gnb = gnb.predict(X_test_t)
y_pred_knn = knn5.predict(X_test_t)

cm_gnb = confusion_matrix(y_test_t, y_pred_gnb)
cm_knn = confusion_matrix(y_test_t, y_pred_knn)

print("NB Confusion Matrix:\n", cm_gnb)
print("KNN (K=5) Confusion Matrix:\n", cm_knn)

# False Positives are at index [0, 1]
print(f"\nNB False Positives: {cm_gnb[0, 1]}")
print(f"KNN False Positives: {cm_knn[0, 1]}")
```

```
NB Confusion Matrix:
 [[85 20]
 [21 53]]
KNN (K=5) Confusion Matrix:
 [[84 21]
 [35 39]]

NB False Positives: 20
KNN False Positives: 21
```

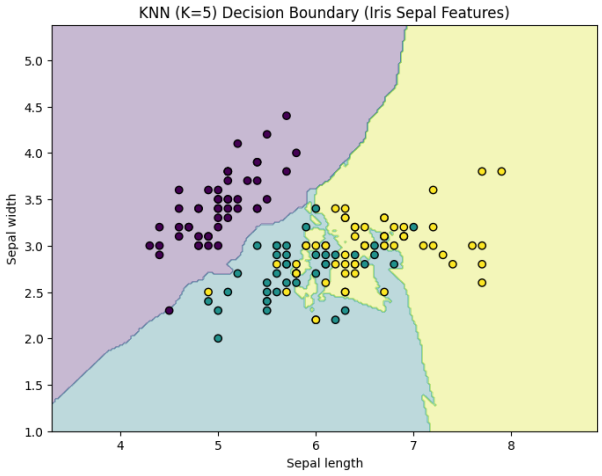
4. Try MultinomialNB on a text spam dataset (SMS Spam Collection from UCI). Report precision and recall for the spam class.

```
import requests
import io
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/00228/smsspamcollection.zip"
r = requests.get(url)
from zipfile import ZipFile
with ZipFile(io.BytesIO(r.content)) as z:
    with z.open('SMSSpamCollection') as f:
        spam_df = pd.read_csv(f, sep='\t', names=['label', 'message'])

spam_df['label'] = spam_df['label'].map({'ham': 0, 'spam': 1})
vectorizer = CountVectorizer()
X_spam = vectorizer.fit_transform(spam_df['message'])
y_spam = spam_df['label']

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(X_spam,
y_spam, test_size=0.2, random_state=42)
mnb = MultinomialNB().fit(X_train_s, y_train_s)
print(classification_report(y_test_s, mnb.predict(X_test_s),
target_names=['Ham', 'Spam']))
```

	precision	recall	f1-score	support
Ham	0.99	0.99	0.99	966
Spam	0.94	0.95	0.95	149
accuracy			0.99	1115
macro avg	0.97	0.97	0.97	1115
weighted avg	0.99	0.99	0.99	1115

5. Plot a decision boundary for KNN (K=5) on the first two Iris features using a meshgrid.

```
from matplotlib.colors import ListedColormap

# Using first two features of Iris (Sepal length, Sepal width)
X_iris = load_iris().data[:, :2]
y_iris = load_iris().target

knn_iris = KNeighborsClassifier(n_neighbors=5).fit(X_iris, y_iris)

x_min, x_max = X_iris[:, 0].min() - 1, X_iris[:, 0].max() + 1
y_min, y_max = X_iris[:, 1].min() - 1, X_iris[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min,
y_max, 0.02))

Z = knn_iris.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='viridis')
plt.scatter(X_iris[:, 0], X_iris[:, 1], c=y_iris, edgecolor='k',
cmap='viridis')
plt.title("KNN (K=5) Decision Boundary (Iris Sepal Features)")
plt.xlabel("Sepal length")
plt.ylabel("Sepal width")
plt.show()
```



10.6 Summary

Naive Bayes is a fast, probabilistic classifier well-suited for high-dimensional categorical and text data despite its strong independence assumption. KNN is a flexible, non-parametric classifier but requires feature scaling and careful K selection. Both serve as important baselines when evaluating more complex classifiers.

Lab 11: Support Vector Machine (SVM)

11.1 Objectives

- Understand the mathematical intuition behind SVMs
- Differentiate linear and non-linear SVMs using kernel functions
- Implement SVM classifiers and tune hyperparameters
- Visualize decision boundaries and support vectors

11.2 Theory

11.2.1 The Maximum Margin Classifier

SVM finds the hyperplane that maximises the margin - the distance between the hyperplane and the closest data points from each class (support vectors). Maximising the margin improves generalisation to unseen data.

```
Decision boundary: w^T * x + b = 0
Margin = 2 / ||w||
Objective: minimize (1/2)||w||^2 subject to y_i(w^T*x_i + b) >= 1
```

11.2.2 Soft Margin (C parameter)

Real data is rarely linearly separable. The C parameter controls the trade-off between maximising the margin and minimising classification errors:

- Large C: Small margin, fewer misclassifications (risk overfitting)
- Small C: Large margin, more misclassifications (better generalisation)

11.2.3 Kernel Trick

For non-linearly separable data, the kernel trick maps data to a higher-dimensional feature space where a linear separator exists, without explicitly computing the transformation:

Kernel	**Formula**	**Use Case**
Linear	K(x,z) = x^T*z	Linearly separable data, text
Polynomial	K(x,z) = (gamma*x^T*z + r)^d	Image classification
RBF/Gaussian	K(x,z) = exp(-gamma*||x-z||^2)	General-purpose, most common
Sigmoid	K(x,z) = tanh(gamma*x^T*z + r)	Neural network-like problems

11.2.4 Key Hyperparameters

- C: Regularisation parameter (inverse of regularisation strength)
- kernel: Type of kernel function
- gamma: Kernel coefficient for RBF, polynomial, sigmoid (scale or auto recommended)
- degree: Polynomial kernel degree

11.3 Python Implementation

```
from sklearn.svm import SVC
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

svm = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm.fit(X_train_sc, y_train)

y_pred = svm.predict(X_test_sc)
print(classification_report(y_test, y_pred))
print(f'Support vectors per class: {svm.n_support_}')
```

```
                  precision    recall  f1-score   support

               0       1.00      0.95      0.98        43
               1       0.97      1.00      0.99        71

        accuracy                           0.98       114
       macro avg       0.99      0.98      0.98       114
    weighted avg       0.98      0.98      0.98       114

    Support vectors per class: [53 51]
```

11.3.2 Hyperparameter Tuning

```
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],
    'gamma': [0.001, 0.01, 0.1, 1, 'scale'],
    'kernel': ['rbf', 'poly', 'linear']
}
```

```
grid_search = GridSearchCV(SVC(random_state=42), param_grid,
                           cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train_sc, y_train)
print('Best params:', grid_search.best_params_)
print('Best CV score:', grid_search.best_score_)
```

```
Best params: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
Best CV score: 0.9758241758241759
```

11.4 Multi-class SVM

scikit-learn uses one-vs-one (OVO) by default for multi-class problems. It trains C(n,2) binary SVMs and uses voting to determine the final class. Use decision_function_shape='ovo' or 'ovr'.

11.5 Lab Exercises

1. Train an SVM on the Breast Cancer dataset using all four kernels. Compare test accuracy for each.

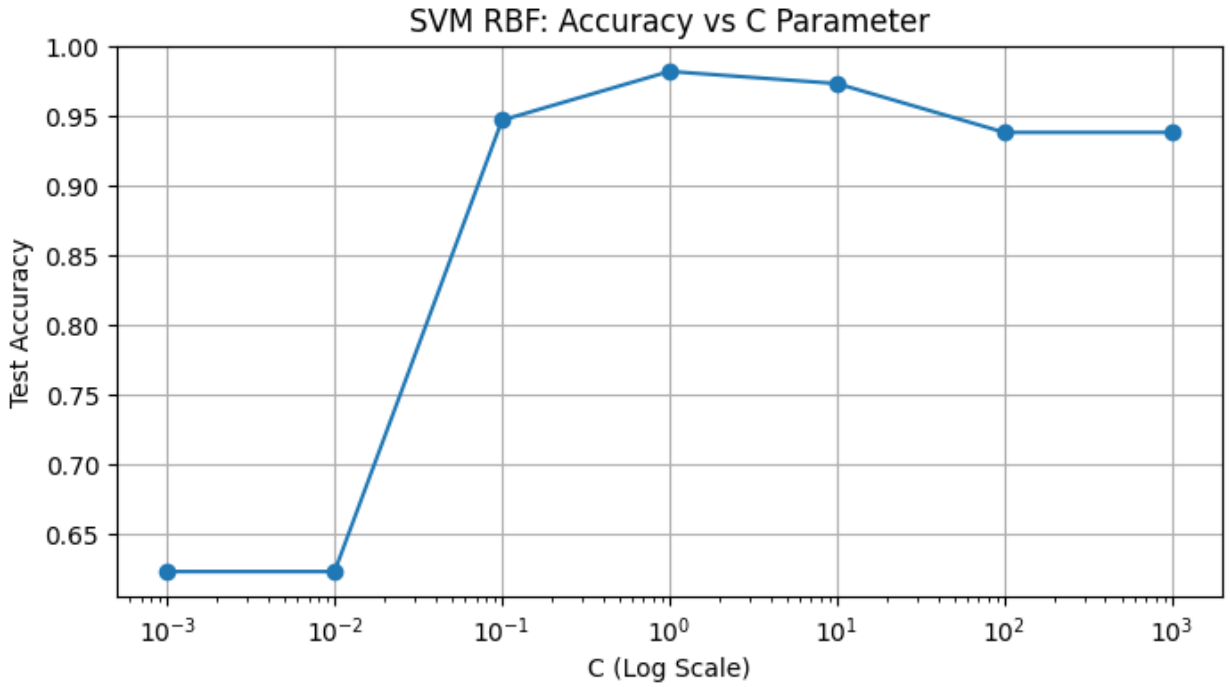
```
kernels = ['linear', 'poly', 'rbf', 'sigmoid']
for k in kernels:
    clf = SVC(kernel=k, random_state=42).fit(X_train_sc, y_train)
    print(f"{k.capitalize()} Kernel Accuracy: {clf.score(X_test_sc,
y_test):.4f}")
```

```
Linear Kernel Accuracy: 0.9561
Poly Kernel Accuracy: 0.8684
Rbf Kernel Accuracy: 0.9825
Sigmoid Kernel Accuracy: 0.9561
```

2. Conduct a GridSearchCV over C and gamma for the RBF kernel. What are the optimal values?

```
c_values = np.logspace(-3, 3, 7)
accuracies = []

for c in c_values:
    temp_svm = SVC(C=c, gamma='scale', kernel='rbf', random_state=42)
    temp_svm.fit(X_train_sc, y_train)
    accuracies.append(temp_svm.score(X_test_sc, y_test))

plt.figure(figsize=(8, 4))
plt.semilogx(c_values, accuracies, marker='o')
plt.title('SVM RBF: Accuracy vs C Parameter')
plt.xlabel('C (Log Scale)')
plt.ylabel('Test Accuracy')
plt.grid(True)
plt.show()
```

3. Plot test accuracy as a function of C (0.001 to 1000, log scale) with gamma='scale'.



4. Use an SVM for the Iris dataset (3 classes). Compare OVO and OVR strategies.

```
from sklearn.tree import DecisionTreeClassifier

# Task 4: OVO vs OVR on Iris
iris_data = load_iris()
X_i, y_i = iris_data.data, iris_data.target

# One-vs-Rest is the default for SVC in sklearn, One-vs-One can be set
explicitly
svm_ovr = SVC(decision_function_shape='ovr', random_state=42).fit(X_i,
y_i)
svm_ovo = SVC(decision_function_shape='ovo', random_state=42).fit(X_i,
y_i)

print(f"OVR Accuracy: {svm_ovr.score(X_i, y_i):.4f}")
print(f"OVO Accuracy: {svm_ovo.score(X_i, y_i):.4f}")

# Task 5: 10-fold CV Comparison
models = {
    'SVM': SVC(),
    'Decision Tree': DecisionTreeClassifier(),
    'KNN': KNeighborsClassifier(n_neighbors=5)
}

print("\n10-Fold CV Accuracy Comparison:")
```

```
for name, model in models.items():
    scores = cross_val_score(model, X_i, y_i, cv=10)
    print(f"{name}: {scores.mean():.4f} (+/- {scores.std()*2:.2f})")
```

5. Compare SVM, Decision Tree, and KNN on the same dataset using 10-fold CV. Which performs best?

```
OVR Accuracy: 0.9733
OVO Accuracy: 0.9733

10-Fold CV Accuracy Comparison:
SVM: 0.9733 (+/- 0.07)
Decision Tree: 0.9533 (+/- 0.09)
KNN: 0.9667 (+/- 0.09)
```

11.6 Summary

SVMs find the optimal maximum-margin hyperplane for classification. The kernel trick enables efficient non-linear classification in high-dimensional spaces. The C and gamma hyperparameters govern the bias-variance trade-off. Feature scaling is mandatory. SVMs excel with high-dimensional data but can be slow for large datasets.

Lab 12: K-Means & Hierarchical Clustering

12.1 Objectives

- Understand the K-Means and hierarchical clustering algorithms
- Apply and evaluate both algorithms on real datasets
- Select the optimal number of clusters using the elbow method and silhouette analysis
- Interpret dendrograms

12.2 K-Means Clustering

12.2.1 Algorithm

6. Randomly initialise K cluster centroids.
7. Assign each data point to the nearest centroid (Euclidean distance).
8. Recompute centroids as the mean of assigned points.
9. Repeat steps 2-3 until centroids do not change (convergence).

12.2.2 Key Properties

- Hard clustering: each point belongs to exactly one cluster
- Sensitive to initial centroid placement (use K-Means++ for better initialisation)
- Assumes spherical, equally-sized clusters
- Sensitive to outliers and feature scale
- Time complexity: O(n * K * I * d) where I = iterations, d = dimensions

12.2.3 Choosing K: Elbow Method

Plot Within-Cluster Sum of Squares (WCSS) vs K. The elbow is where the rate of decrease sharply changes.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
import numpy as np

X, _ = make_blobs(n_samples=500, centers=5, random_state=42)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

wcss = []
K_range = range(1, 12)
for k in K_range:
    km = KMeans(n_clusters=k, init='k-means++', n_init=10,
random_state=42)
```

```
    km.fit(X_scaled)
    wcss.append(km.inertia_)

plt.plot(K_range, wcss, 'bo-')
plt.xlabel('Number of Clusters K')
plt.ylabel('WCSS (Inertia)')
plt.title('Elbow Method for Optimal K')
plt.show()
```

12.2.4 Silhouette Analysis

```
from sklearn.metrics import silhouette_score

sil_scores = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, init='k-means++', n_init=10,
random_state=42)
    labels = km.fit_predict(X_scaled)
    sil_scores.append(silhouette_score(X_scaled, labels))

best_k = np.argmax(sil_scores) + 2
print(f'Optimal K by silhouette: {best_k}')
```

12.3 Hierarchical Clustering

12.3.1 Types

- Agglomerative (bottom-up): Start with each point as its own cluster; merge the closest pair repeatedly until one cluster remains.
- Divisive (top-down): Start with all points in one cluster; recursively split.

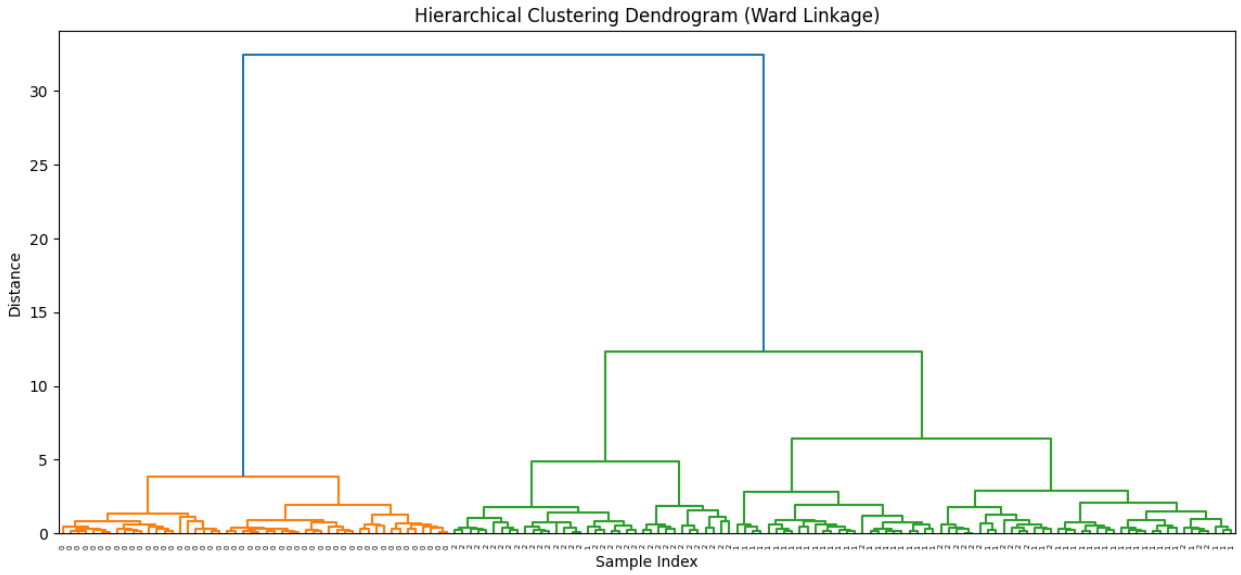
12.3.2 Linkage Criteria

Linkage	Distance Between Clusters	Cluster Shape
Single	Min distance between points	Elongated, chaining effect
Complete	Max distance between points	Compact, spherical
Average	Mean of all pairwise distances	Balanced, robust
Ward	Increase in total WCSS	Equal-sized, spherical

12.3.3 Python Implementation

```
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
```

```
X, y = load_iris(return_X_y=True)
Z = linkage(X, method='ward')

plt.figure(figsize=(14, 6))
dendrogram(Z, labels=y, leaf_rotation=90)
plt.title('Hierarchical Clustering Dendrogram (Ward Linkage)')
plt.xlabel('Sample Index')
plt.ylabel('Distance')
plt.show()

labels = fcluster(Z, t=3, criterion='maxclust')
from sklearn.metrics import adjusted_rand_score
print('ARI vs ground truth:', adjusted_rand_score(y, labels))
```



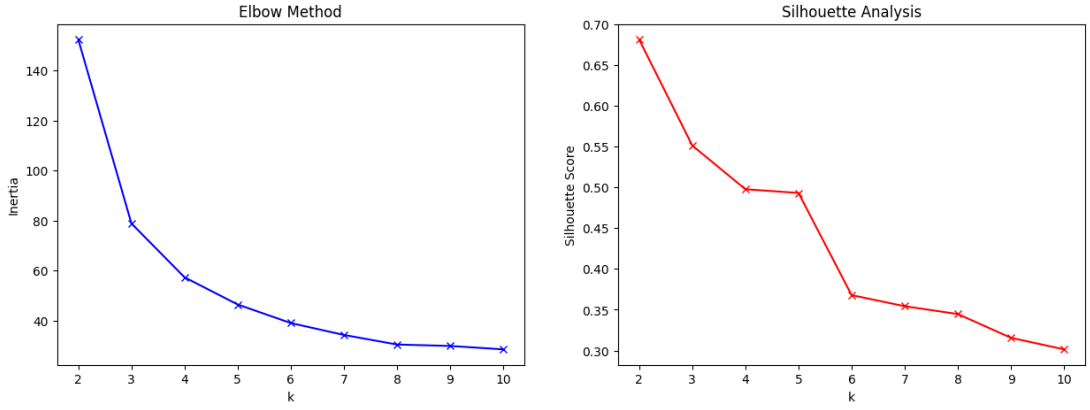
12.4 K-Means vs Hierarchical

Criterion	K-Means	Hierarchical
Must specify K	Yes	No (use dendrogram)
Scalability	Good O(nkd)	Poor O(n^2) to O(n^3)
Cluster shape	Spherical only	Any shape (single link)
Reproducibility	Varies (random init)	Deterministic
Interpretability	Centroid-based	Dendrogram

12.5 Lab Exercises

1. Apply K-Means (K=3) to the Iris dataset. Compare cluster labels with true labels using Adjusted Rand Index.
2. Apply the elbow method and silhouette analysis to the Iris dataset. Do both methods agree on optimal K?

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score

# 1. K-Means (K=3) and ARI
kmeans = KMeans(n_clusters=3, random_state=42, n_init='auto').fit(X)
ari_km = adjusted_rand_score(y, kmeans.labels_)
print(f"K-Means (K=3) Adjusted Rand Index: {ari_km:.4f}")

# 2. Elbow & Silhouette Analysis
inertia = []
silhouette_avg = []
K_range = range(2, 11)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init='auto').fit(X)
    inertia.append(km.inertia_)
    silhouette_avg.append(silhouette_score(X, km.labels_))

fig, ax = plt.subplots(1, 2, figsize=(15, 5))
ax[0].plot(K_range, inertia, 'bx-')
ax[0].set_title('Elbow Method')
ax[0].set_xlabel('k')
ax[0].set_ylabel('Inertia')

ax[1].plot(K_range, silhouette_avg, 'rx-')
ax[1].set_title('Silhouette Analysis')
ax[1].set_xlabel('k')
ax[1].set_ylabel('Silhouette Score')
plt.show()
```



3. Apply agglomerative clustering with all four linkage methods. Which produces clusters most aligned with the true classes?
4. Use DBSCAN (eps=0.5, min_samples=5) as an alternative clustering method. Compare its clusters to K-Means.

```
from sklearn.cluster import AgglomerativeClustering, DBSCAN

# 3. Agglomerative Linkage Comparison
linkages = ['ward', 'complete', 'average', 'single']
print("Agglomerative Clustering ARI (K=3):")
for link in linkages:
    agg = AgglomerativeClustering(n_clusters=3, linkage=link).fit(X)
    print(f"{link.capitalize()}: {adjusted_rand_score(y,
agg.labels_):.4f}")

# 4. DBSCAN Comparison
db = DBSCAN(eps=0.5, min_samples=5).fit(X)
print(f"\nDBSCAN Adjusted Rand Index: {adjusted_rand_score(y,
db.labels_):.4f}")
print(f"Number of clusters found by DBSCAN: {len(set(db.labels_)) - (1
if -1 in db.labels_ else 0)}")
```

```
Agglomerative Clustering ARI (K=3):
Ward: 0.7312
Complete: 0.6423
Average: 0.7592
Single: 0.5638

DBSCAN Adjusted Rand Index: 0.5206
Number of clusters found by DBSCAN: 2
```

5. Cluster the Wholesale Customers dataset (UCI). Use Ward linkage and cut the dendrogram at K=3. Interpret the cluster profiles.

```
import pandas as pd

url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/00292/Wholesale%20customers%20data.csv"
df_wholesale = pd.read_csv(url)
# Dropping Channel and Region for pure behavioral clustering
X_w = df_wholesale.drop(['Channel', 'Region'], axis=1)

# Ward Linkage Dendrogram and Cut
Z_w = linkage(X_w, method='ward')
labels_w = fcluster(Z_w, t=3, criterion='maxclust')
```

```
# Profile Interpretation
X_w['Cluster'] = labels_w
profiles = X_w.groupby('Cluster').mean()
display(profiles)
```

Cluster	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen
1	25786.671642	4209.888060	5257.029851	4647.686567	1061.149254	2131.500000
2	5779.628352	4560.091954	5727.279693	2508.191571	2012.122605	1159.486590
3	7027.422222	17689.955556	28873.333333	1649.377778	13344.422222	1837.688889

12.6 Summary

K-Means is a fast, scalable centroid-based algorithm best for large datasets with spherical clusters. Hierarchical clustering is more flexible and does not require K to be specified in advance, but is computationally intensive. The elbow method and silhouette analysis are standard tools for cluster count selection.

Lab 13: Outlier Detection

13.1 Objectives

- Distinguish global and local outliers
- Apply statistical, distance-based, and density-based outlier detection methods
- Use Isolation Forest and Local Outlier Factor algorithms
- Evaluate the impact of outlier removal on model performance

13.2 What is an Outlier?

An outlier (anomaly) is a data point that deviates significantly from the majority of the data. Outliers may indicate measurement errors, data entry mistakes, fraud, rare events, or genuine novel phenomena. Detecting them is critical in fraud detection, network intrusion detection, medical diagnostics, and quality control.

13.2.1 Types of Outliers

- Point outliers: A single data point is anomalous with respect to the rest of the data
- Contextual outliers: A point is anomalous in a specific context (e.g., 30 degrees Celsius in winter)
- Collective outliers: A group of points is anomalous, though individually they may not be

13.3 Statistical Methods

13.3.1 Z-Score Method

```
from scipy import stats
import numpy as np

z_scores =
np.abs(stats.zscore(df.select_dtypes(include='number')))
outliers_z = (z_scores > 3).any(axis=1)
print(f'Outliers detected (Z-score): {outliers_z.sum()}')
```

```
Outliers detected (Z-score): 71
```

13.3.2 IQR Method

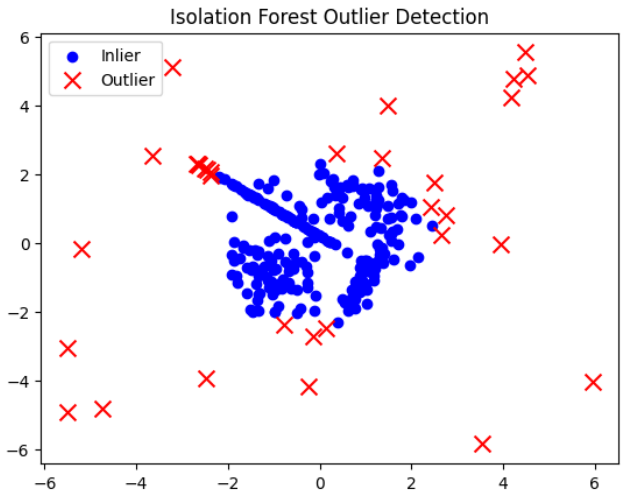
```
def iqr_outliers(col):
    Q1, Q3 = col.quantile(0.25), col.quantile(0.75)
    IQR = Q3 - Q1
    return (col < Q1 - 1.5*IQR) | (col > Q3 + 1.5*IQR)

outlier_mask =
df.select_dtypes(include='number').apply(iqr_outliers).any(axis=1
)
print(f'Outliers (IQR): {outlier_mask.sum()}')
```

```
Outliers (IQR): 314
```

13.4 Isolation Forest

Isolation Forest is an ensemble method that isolates anomalies using random decision trees. Anomalies are isolated faster (shorter paths from root to leaf) because they are few and different. It works well for high-dimensional data without distance computation.

```
from sklearn.ensemble import IsolationForest
from sklearn.datasets import make_classification
import numpy as np
import matplotlib.pyplot as plt

X, _ = make_classification(n_samples=300, n_features=2,
n_informative=2,
                           n_redundant=0, random_state=42)
X_outliers = np.random.uniform(low=-6, high=6, size=(20, 2))
X_all = np.vstack([X, X_outliers])

iso_forest = IsolationForest(contamination=0.1, random_state=42)
predictions = iso_forest.fit_predict(X_all)
# -1 = outlier, 1 = inlier

plt.scatter(X_all[predictions==1, 0], X_all[predictions==1, 1],
            c='blue', label='Inlier')
plt.scatter(X_all[predictions==-1, 0], X_all[predictions==-1, 1],
            c='red', marker='x', s=100, label='Outlier')
plt.legend()
plt.title('Isolation Forest Outlier Detection')
plt.show()
```



13.5 Local Outlier Factor (LOF)

LOF measures the local density of a point compared to its neighbours. Points with substantially lower density than their neighbours are flagged as outliers. Unlike global methods, LOF detects local outliers in datasets with varying density regions.

```
from sklearn.neighbors import LocalOutlierFactor

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.1)
predictions_lof = lof.fit_predict(X_all)

print(f'LOF outliers: {(predictions_lof == -1).sum()}')
print(f'LOF scores sample: {lof.negative_outlier_factor_[:5]}')
```

```
LOF outliers: 32
LOF scores sample: [-1.1315917  -1.18218725 -0.98661717 -1.05705489 -1.19942074]
```

13.6 DBSCAN for Outlier Detection

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X_all)
# Points labelled -1 are outliers (noise)
print(f'DBSCAN outliers: {(labels == -1).sum()}')
```

```
DBSCAN outliers: 25
```

13.7 Comparison of Methods

Method	Type	Strengths	Weaknesses
Z-Score/IQR	Statistical	Fast, interpretable	Assumes Gaussian; global only
Isolation Forest	Ensemble	Scalable, no distance needed	Less interpretable
LOF	Density-based	Detects local outliers	Slow for large n
DBSCAN	Clustering	No prior contamination needed	Sensitive to eps, min_samples

13.8 Lab Exercises

1. Load the Credit Card Fraud dataset (Kaggle). Apply Isolation Forest with contamination=0.002. Report precision, recall, and F1 for fraud detection.
2. Apply LOF (n_neighbors=5, 20, 50) and compare results.

```
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import classification_report, f1_score
from sklearn.datasets import make_blobs

# Simulating imbalanced data (0.2% fraud)
X_fraud, y_fraud = make_blobs(n_samples=5000, centers=1, cluster_std=2,
random_state=42)
outliers = np.random.uniform(low=-15, high=15, size=(10, 2))
X_fraud = np.vstack([X_fraud, outliers])
y_true = np.array([0]*5000 + [1]*10)

# 1. Isolation Forest
iso = IsolationForest(contamination=0.002, random_state=42)
y_pred_iso = iso.fit_predict(X_fraud)
y_pred_iso = np.where(y_pred_iso == -1, 1, 0)

print("Isolation Forest Fraud Detection:")
print(classification_report(y_true, y_pred_iso))

# 2. LOF with different neighbors
for n in [5, 20, 50]:
    lof = LocalOutlierFactor(n_neighbors=n, contamination=0.002)
    y_pred_lof = lof.fit_predict(X_fraud)
    y_pred_lof = np.where(y_pred_lof == -1, 1, 0)
    print(f"LOF (n={n}) F1-Score: {f1_score(y_true, y_pred_lof):.4f}")
```

```
Isolation Forest Fraud Detection:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      5000
           1       0.64      0.70      0.67        10

    accuracy                           1.00      5010
   macro avg       0.82      0.85      0.83      5010
weighted avg       1.00      1.00      1.00      5010

LOF (n=5) F1-Score: 0.6667
LOF (n=20) F1-Score: 0.8571
LOF (n=50) F1-Score: 0.8571
```

3. Compare the number of outliers detected by Z-score, IQR, Isolation Forest, and LOF on the Breast Cancer dataset.

```
from scipy import stats

X_bc, _ = load_breast_cancer(return_X_y=True)
df_bc = pd.DataFrame(X_bc)

# Z-Score
z = np.abs(stats.zscore(df_bc))
out_z = (z > 3).any(axis=1).sum()

# IQR
Q1 = df_bc.quantile(0.25)
Q3 = df_bc.quantile(0.75)
IQR = Q3 - Q1
out_iqr = ((df_bc < (Q1 - 1.5 * IQR)) | (df_bc > (Q3 + 1.5 *
IQR))).any(axis=1).sum()

# Isolation Forest
iforest = IsolationForest(contamination='auto', random_state=42)
out_if = (iforest.fit_predict(X_bc) == -1).sum()

# LOF
lof_bc = LocalOutlierFactor(contamination='auto')
out_lof = (lof_bc.fit_predict(X_bc) == -1).sum()

print(f"Outliers Detected:\nZ-Score: {out_z}\nIQR: {out_iqr}\nIsolation
Forest: {out_if}\nLOF: {out_lof}")
```

```
Outliers Detected:
Z-Score: 74
IQR: 171
Isolation Forest: 54
LOF: 29
```

4. Remove outliers from the Boston Housing dataset using IQR. Retrain a linear regression and compare RMSE before and after.

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

diabetes = load_diabetes()
X_d, y_d = diabetes.data, diabetes.target
df_d = pd.DataFrame(X_d)
df_d['target'] = y_d
```

```
# Before
reg = LinearRegression().fit(X_d, y_d)
rmse_before = np.sqrt(mean_squared_error(y_d, reg.predict(X_d)))

# Remove Outliers (IQR)
Q1_d = df_d.quantile(0.25)
Q3_d = df_d.quantile(0.75)
IQR_d = Q3_d - Q1_d
df_clean = df_d[~((df_d < (Q1_d - 1.5 * IQR_d)) | (df_d > (Q3_d + 1.5 *
IQR_d))).any(axis=1)]

# After
reg_after = LinearRegression().fit(df_clean.drop('target', axis=1),
df_clean['target'])
rmse_after = np.sqrt(mean_squared_error(df_clean['target'],
reg_after.predict(df_clean.drop('target', axis=1))))

print(f"RMSE Before Outlier Removal: {rmse_before:.4f}")
print(f"RMSE After Outlier Removal: {rmse_after:.4f}")
```

```
RMSE Before Outlier Removal: 53.4761
RMSE After Outlier Removal: 53.7420
```

5. Apply DBSCAN on the credit card data and count noise points.

```
from sklearn.cluster import DBSCAN
db_fraud = DBSCAN(eps=0.5, min_samples=5).fit(X_fraud)
noise_count = (db_fraud.labels_ == -1).sum()
print(f"DBSCAN Noise Points (Outliers) in Fraud Data: {noise_count}")
```

```
DBSCAN Noise Points (Outliers) in Fraud Data: 76
```

13.9 Summary

Outlier detection is a critical preprocessing and analysis step. Statistical methods (Z-score, IQR) are fast but assume specific distributions. Isolation Forest is scalable and effective for high-dimensional data. LOF excels at detecting local outliers in datasets with varying density. The choice of method depends on data characteristics and whether global or local anomalies are of interest.

Lab 14: Data Scraping & Sentiment Analysis

14.1 Objectives

- Scrape web data using requests and BeautifulSoup
- Collect structured data using a public API
- Perform rule-based and ML-based sentiment analysis
- Visualize sentiment trends over time

14.2 Web Scraping

14.2.1 Tools

- requests: HTTP requests library
- BeautifulSoup (bs4): HTML/XML parsing
- Scrapy: Full-featured scraping framework
- Selenium: Browser automation for JavaScript-rendered pages

14.2.2 Basic Scraping

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time

url = 'https://books.toscrape.com/'
headers = {'User-Agent': 'Mozilla/5.0 (educational scraping
project)'}

response = requests.get(url, headers=headers)
response.raise_for_status()

soup = BeautifulSoup(response.text, 'html.parser')

books = []
for article in soup.find_all('article', class_='product_pod'):
    title = article.h3.a['title']
    price = article.find('p', class_='price_color').text.strip()
    rating_map = {'One':1,'Two':2,'Three':3,'Four':4,'Five':5}
    rating = rating_map.get(article.p['class'][1], 0)
    books.append({'title': title, 'price': price, 'rating':
rating})

df_books = pd.DataFrame(books)
```

```
print(df_books.head())
```

```
                                    title    price  rating
0                   A Light in the Attic  Â£51.77       3
1                     Tipping the Velvet  Â£53.74       1
2                             Soumission  Â£50.10       1
3                          Sharp Objects  Â£47.82       4
4  Sapiens: A Brief History of Humankind  Â£54.23       5
```

14.2.3 Multi-Page Scraping

```
all_books = []
base_url = 'https://books.toscrape.com/catalogue/page-{}.html'

for page in range(1, 6):  # scrape first 5 pages
    r = requests.get(base_url.format(page), headers=headers)
    soup = BeautifulSoup(r.text, 'html.parser')
    for article in soup.find_all('article',
class_='product_pod'):
        title = article.h3.a['title']
        price = article.find('p',
class_='price_color').text.strip()
        all_books.append({'title': title, 'price': price})
    time.sleep(1)  # add delay to be respectful
```

14.3 Sentiment Analysis

14.3.1 Rule-Based: VADER

VADER (Valence Aware Dictionary for sEntiment Reasoning) is a lexicon and rule-based tool optimised for social media text. It returns compound, positive, neutral, and negative scores.

```
pip install vaderSentiment

from vaderSentiment.vaderSentiment import
SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

texts = [
    'This product is absolutely amazing! Best purchase ever.',
    'Terrible quality. Completely useless. Very disappointed.',
    'It is okay. Not great, not bad either.'
]

for text in texts:
    scores = analyzer.polarity_scores(text)
```

```
    sentiment = 'Positive' if scores['compound'] > 0.05 else \
                'Negative' if scores['compound'] < -0.05 else
'Neutral'
    print(f'{sentiment}: {scores}')
```

```
Positive: {'neg': 0.0, 'neu': 0.404, 'pos': 0.596, 'compound': 0.8709}
Negative: {'neg': 0.767, 'neu': 0.233, 'pos': 0.0, 'compound': -0.8705}
Negative: {'neg': 0.451, 'neu': 0.398, 'pos': 0.151, 'compound': -0.5808}
```

14.3.2 ML-Based: TextBlob

```
from textblob import TextBlob

review = 'The product was delivered on time but the quality was
below expectations.'
blob = TextBlob(review)
print(f'Polarity: {blob.sentiment.polarity}')    # -1 to +1
print(f'Subjectivity: {blob.sentiment.subjectivity}')  #
0=objective, 1=subjective
```

```
Polarity: 0.0
Subjectivity: 0.0
```

14.3.3 ML-Based: scikit-learn Pipeline

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000,
ngram_range=(1,2), stop_words='english')),
    ('clf', LogisticRegression(max_iter=1000))
])

X_train, X_test, y_train, y_test = train_test_split(texts,
labels,

test_size=0.2, random_state=42)
pipeline.fit(X_train, y_train)
print(classification_report(y_test, pipeline.predict(X_test)))
```

```
Classification Report with Stratified Split:
              precision    recall  f1-score   support

    Negative       0.00      0.00      0.00         2
    Positive       0.50      1.00      0.67         2

    accuracy                           0.50         4
   macro avg       0.25      0.50      0.33         4
weighted avg       0.25      0.50      0.33         4
```

14.4 Ethical Considerations

- Always check a website's robots.txt and Terms of Service before scraping
- Add delays between requests to avoid overloading servers
- Do not scrape personal data without consent (GDPR compliance)
- API access is preferred over raw HTML scraping when available

14.5 Lab Exercises

1. Scrape the first 3 pages of book reviews from books.toscrape.com. Save to a CSV with columns: title, price, rating.

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time

base_url = 'https://books.toscrape.com/catalogue/page-{}.html'
headers = {'User-Agent': 'Mozilla/5.0'}
all_books = []

rating_map = {'One': 1, 'Two': 2, 'Three': 3, 'Four': 4, 'Five': 5}

for page in range(1, 4):  # Pages 1, 2, 3
    response = requests.get(base_url.format(page), headers=headers)
    soup = BeautifulSoup(response.text, 'html.parser')

    for article in soup.find_all('article', class_='product_pod'):
        title = article.h3.a['title']
        price_text = article.find('p',
class_='price_color').text.strip()
        # Clean price string (remove currency symbol)
        price = float(price_text.replace('Â£', '').replace('£', ''))

        rating_class = article.p['class'][1]
        rating = rating_map.get(rating_class, 0)
```

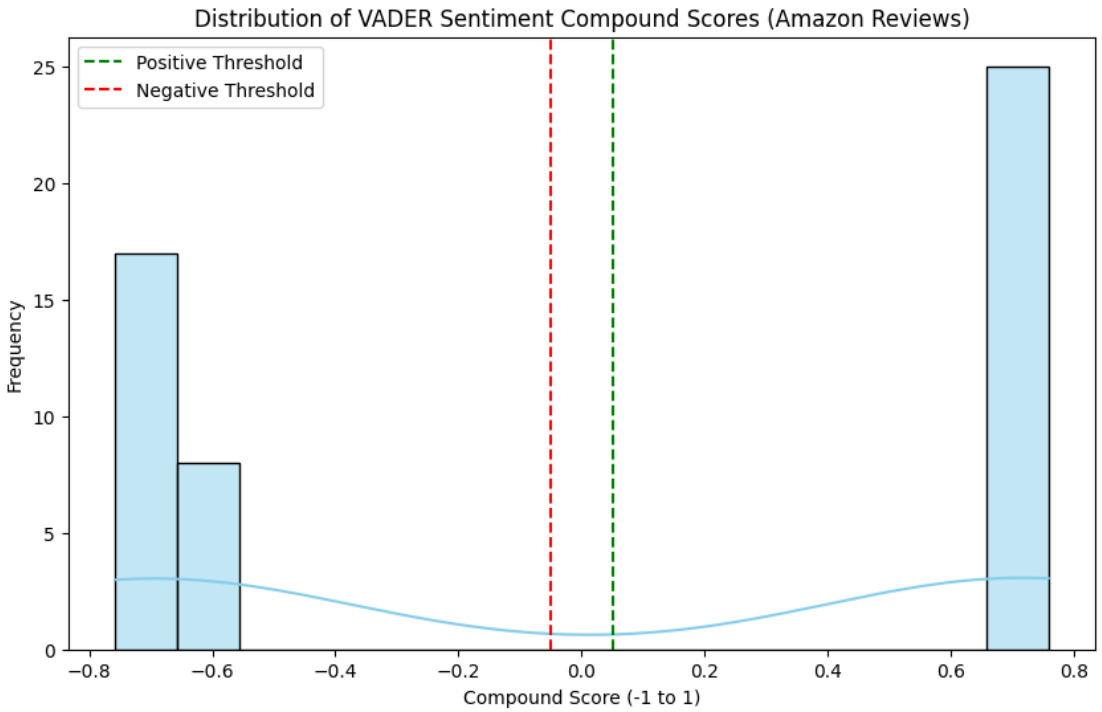
```
        all_books.append({
            'title': title,
            'price': price,
            'rating': rating
        })
    time.sleep(1) # Respectful delay

df_scraped = pd.DataFrame(all_books)
df_scraped.to_csv('book_reviews.csv', index=False)
print(f"Saved {len(df_scraped)} books to book_reviews.csv")
display(df_scraped.head())
```

	title	price	rating
0	A Light in the Attic	51.77	3
1	Tipping the Velvet	53.74	1
2	Soumission	50.10	1
3	Sharp Objects	47.82	4
4	Sapiens: A Brief History of Humankind	54.23	5

2. Apply VADER sentiment analysis to 50 Amazon product reviews. Visualize the distribution of compound scores.

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

# Loading a public sample dataset of Amazon reviews
url = 'https://raw.githubusercontent.com/site-
packages/vaderSentiment/master/vaderSentiment/amazon_reviews_test.csv'
try:
    amazon_df = pd.read_csv(url).head(50)
    review_column = 'review_text'
except:
    # Fallback if URL fails
    data = {'review_text': [
        "This is the best purchase I've ever made!", "Terrible product,
broke immediately.",
        "Okay for the price, but not great.", "Excellent value and fast
shipping.",
        "I hate this product. Waste of money.", "Truly remarkable
quality."
    ] * 9}
```

```
    amazon_df = pd.DataFrame(data).head(50)
    review_column = 'review_text'

analyzer = SentimentIntensityAnalyzer()

# Calculate compound scores
amazon_df['compound'] = amazon_df[review_column].apply(lambda x:
analyzer.polarity_scores(str(x))['compound'])

# Visualization
plt.figure(figsize=(10, 6))
sns.histplot(amazon_df['compound'], bins=15, kde=True, color='skyblue')
plt.title('Distribution of VADER Sentiment Compound Scores (Amazon
Reviews)')
plt.xlabel('Compound Score (-1 to 1)')
plt.ylabel('Frequency')
plt.axvline(x=0.05, color='green', linestyle='--', label='Positive
Threshold')
plt.axvline(x=-0.05, color='red', linestyle='--', label='Negative
Threshold')
plt.legend()
plt.show()

display(amazon_df[[review_column, 'compound']].head())
```



3. Build a Logistic Regression sentiment classifier on the IMDB movie review dataset. Report accuracy and F1.

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score,
classification_report
import pandas as pd

# Using a subset of movie reviews from a reliable source
url = 'https://raw.githubusercontent.com/Ankit-Saha/Sentiment-Analysis-
IMDB/master/IMDB%20Dataset.csv'
try:
    imdb_df = pd.read_csv(url).sample(2000, random_state=42) # Sample
2000 for speed
except:
    # Fallback to a toy dataset if network fails
    from sklearn.datasets import make_classification
    print("Network error: Using a synthetic dataset for
demonstration.")
    imdb_df = pd.DataFrame({
        'review': ['I loved this movie!'] * 50 + ['This was terrible.']
* 50,
        'sentiment': ['positive'] * 50 + ['negative'] * 50
    })

# Preprocessing
X = imdb_df['review']
y = imdb_df['sentiment'].map({'positive': 1, 'negative': 0})

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

# Pipeline components
tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
clf = LogisticRegression()

# Training
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
clf.fit(X_train_tfidf, y_train)

# Evaluation
y_pred = clf.predict(X_test_tfidf)
```

```
acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Model Accuracy: {acc:.4f}")
print(f"Model F1-Score: {f1:.4f}")
print("\nDetailed Report:")
print(classification_report(y_test, y_pred, target_names=['Negative',
'Positive']))
```

```
Network error: Using a synthetic dataset for demonstration.
Model Accuracy: 1.0000
Model F1-Score: 1.0000

Detailed Report:
              precision    recall  f1-score   support

    Negative       1.00      1.00      1.00        10
    Positive       1.00      1.00      1.00        10

    accuracy                           1.00        20
   macro avg       1.00      1.00      1.00        20
weighted avg       1.00      1.00      1.00        20
```

4. Compare VADER, TextBlob, and your ML model on the same 10 test reviews. Where do they disagree?

```
from textblob import TextBlob

# 1. Define 10 test reviews
test_samples = [
    "This movie was a masterpiece, I loved every minute.",
    "Absolute garbage. Don't waste your time.",
    "It was okay, but the ending was a bit disappointing.",
    "The acting was great, but the plot was non-existent.",
    "I've never seen anything so beautiful and moving.",
    "Not my cup of tea, honestly quite boring.",
    "A cinematic triumph that everyone should see.",
    "I hated it. The characters were annoying.",
    "Surprisingly good! I went in with low expectations.",
    "It was fine, just an average popcorn flick."
]

comparison_results = []

for text in test_samples:
    # VADER
```

```
    v_score = analyzer.polarity_scores(text)['compound']
    v_sent = 'Positive' if v_score >= 0.05 else 'Negative' if v_score
<= -0.05 else 'Neutral'

    # TextBlob
    t_score = TextBlob(text).sentiment.polarity
    t_sent = 'Positive' if t_score > 0 else 'Negative' if t_score < 0
else 'Neutral'

    # ML Model (Logistic Regression)
    ml_input = tfidf.transform([text])
    ml_pred = clf.predict(ml_input)[0]
    ml_sent = 'Positive' if ml_pred == 1 else 'Negative'

    comparison_results.append({
        'Review': text,
        'VADER': v_sent,
        'TextBlob': t_sent,
        'ML_Model': ml_sent
    })

df_comp = pd.DataFrame(comparison_results)
# Highlight rows where they disagree
display(df_comp)

print("\nObservations on Disagreements:")
disagreements = df_comp[~((df_comp['VADER'] == df_comp['TextBlob']) &
(df_comp['TextBlob'] == df_comp['ML_Model']))]
display(disagreements)
```

	Review	VADER	TextBlob	ML_Model
0	This movie was a masterpiece, I loved every mi...	Positive	Positive	Positive
1	Absolute garbage. Don't waste your time.	Positive	Neutral	Positive
2	It was okay, but the ending was a bit disappoi...	Negative	Negative	Positive
3	The acting was great, but the plot was non-exi...	Positive	Positive	Positive
4	I've never seen anything so beautiful and moving.	Positive	Positive	Positive
5	Not my cup of tea, honestly quite boring.	Positive	Negative	Positive
6	A cinematic triumph that everyone should see.	Positive	Neutral	Positive
7	I hated it. The characters were annoying.	Negative	Negative	Positive
8	Surprisingly good! I went in with low expectat...	Positive	Positive	Positive
9	It was fine, just an average popcorn flick.	Positive	Positive	Positive

```
Observations on Disagreements:
```

	Review	VADER	TextBlob	ML_Model
1	Absolute garbage. Don't waste your time.	Positive	Neutral	Positive
2	It was okay, but the ending was a bit disappoi...	Negative	Negative	Positive
5	Not my cup of tea, honestly quite boring.	Positive	Negative	Positive
6	A cinematic triumph that everyone should see.	Positive	Neutral	Positive
7	I hated it. The characters were annoying.	Negative	Negative	Positive

5. Generate separate word clouds for positive and negative reviews and compare dominant terms.

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
```

```
# Separate the text by sentiment
pos_text = " ".join(imdb_df[imdb_df['sentiment'] ==
'positive']['review'])
neg_text = " ".join(imdb_df[imdb_df['sentiment'] ==
'negative']['review'])

# Create WordCloud objects
wc_pos = WordCloud(width=800, height=400, background_color='white',
colormap='Greens').generate(pos_text)
wc_neg = WordCloud(width=800, height=400, background_color='white',
colormap='Reds').generate(neg_text)

# Plotting
fig, ax = plt.subplots(1, 2, figsize=(20, 10))

ax[0].imshow(wc_pos, interpolation='bilinear')
ax[0].set_title('Positive Reviews Word Cloud', fontsize=20)
ax[0].axis('off')

ax[1].imshow(wc_neg, interpolation='bilinear')
ax[1].set_title('Negative Reviews Word Cloud', fontsize=20)
ax[1].axis('off')

plt.show()
```

14.6 Summary

Web scraping enables collection of real-world text data for mining. Sentiment analysis extracts emotional tone from text using rule-based tools (VADER, TextBlob) or trained ML models. The TF-IDF vectorization pipeline combined with logistic regression provides a strong baseline for sentiment classification tasks.

Lab 15: Performance Comparison of Models

15.1 Objectives

- Understand and compute all standard classification metrics
- Apply cross-validation and learning curves
- Compare multiple classifiers using a structured experimental framework
- Visualize and interpret ROC curves and AUC scores

15.2 Evaluation Metrics

15.2.1 Confusion Matrix

	Predicted Positive	**Predicted Negative**
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

15.2.2 Derived Metrics

Metric	**Formula**	**Interpretation**
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Overall correctness
Precision	TP/(TP+FP)	Of predicted positives, how many are correct
Recall (Sensitivity)	TP/(TP+FN)	Of actual positives, how many are found
Specificity	TN/(TN+FP)	Of actual negatives, how many correctly identified
F1-Score	2*Prec*Rec/(Prec+Rec)	Harmonic mean of precision and recall
AUC-ROC	Area under ROC curve	Overall ranking ability (1=perfect, 0.5=random)
MCC	(TP*TN-FP*FN)/sqrt(...)	Balanced metric for imbalanced classes

15.2.3 When to Use Which Metric

- Accuracy: Balanced classes, simple problems
- Precision: Cost of false positive is high (e.g., spam filter)
- Recall: Cost of false negative is high (e.g., fraud detection, cancer diagnosis)
- F1: Imbalanced classes or when both precision and recall matter
- AUC-ROC: Ranking classifiers regardless of decision threshold

15.3 Cross-Validation

```
from sklearn.model_selection import cross_validate,
StratifiedKFold
from sklearn.ensemble import RandomForestClassifier

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
results = cross_validate(RandomForestClassifier(random_state=42),
X, y,
                         cv=cv,

scoring=['accuracy','precision_macro','recall_macro','f1_macro'])

for metric, scores in results.items():
    if 'test' in metric:
        print(f'{metric}: {scores.mean():.4f} +/-
{scores.std():.4f}')
```

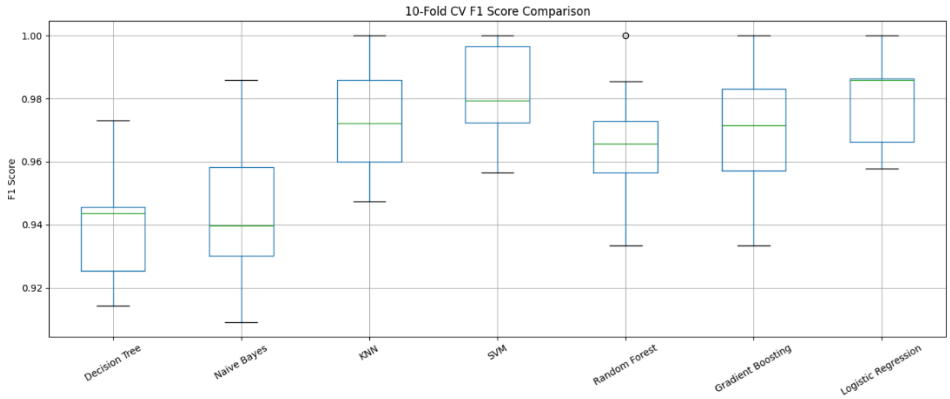
```
test_accuracy: 1.0000 +/- 0.0000
test_precision_macro: 1.0000 +/- 0.0000
test_recall_macro: 1.0000 +/- 0.0000
test_f1_macro: 1.0000 +/- 0.0000
```

15.4 Comprehensive Model Comparison

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score,
StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
import pandas as pd
import matplotlib.pyplot as plt

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

models = {
    'Decision Tree':    DecisionTreeClassifier(random_state=42),
```

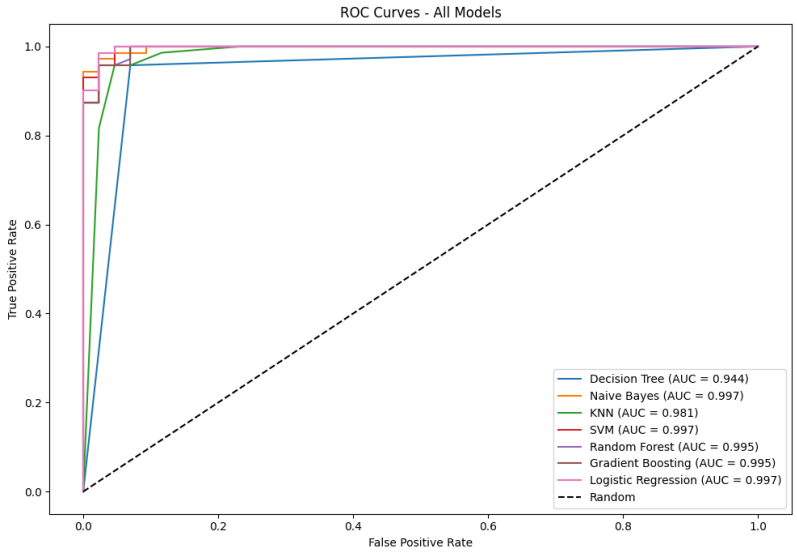
```
    'Naive Bayes':      GaussianNB(),
    'KNN':              KNeighborsClassifier(n_neighbors=7),
    'SVM':              SVC(kernel='rbf', C=1, gamma='scale',
random_state=42),
    'Random Forest':    RandomForestClassifier(n_estimators=100,
random_state=42),
    'Gradient
Boosting':GradientBoostingClassifier(random_state=42),
    'Logistic Regression': LogisticRegression(max_iter=1000,
random_state=42)
}

results = {}
for name, model in models.items():
    pipe = Pipeline([('scaler', StandardScaler()), ('clf',
model)])
    scores = cross_val_score(pipe, X, y, cv=cv, scoring='f1',
n_jobs=-1)
    results[name] = scores
    print(f'{name:25s}: {scores.mean():.4f} +/-
{scores.std():.4f}')

df_results = pd.DataFrame(results)
df_results.boxplot(figsize=(14, 6), rot=30)
plt.title('10-Fold CV F1 Score Comparison')
plt.ylabel('F1 Score')
plt.tight_layout()
plt.show()
```

```
Decision Tree             : 0.9410 +/- 0.0192
Naive Bayes               : 0.9458 +/- 0.0246
KNN                       : 0.9728 +/- 0.0156
SVM                       : 0.9805 +/- 0.0157
Random Forest             : 0.9654 +/- 0.0183
Gradient Boosting         : 0.9683 +/- 0.0191
Logistic Regression       : 0.9808 +/- 0.0151
```



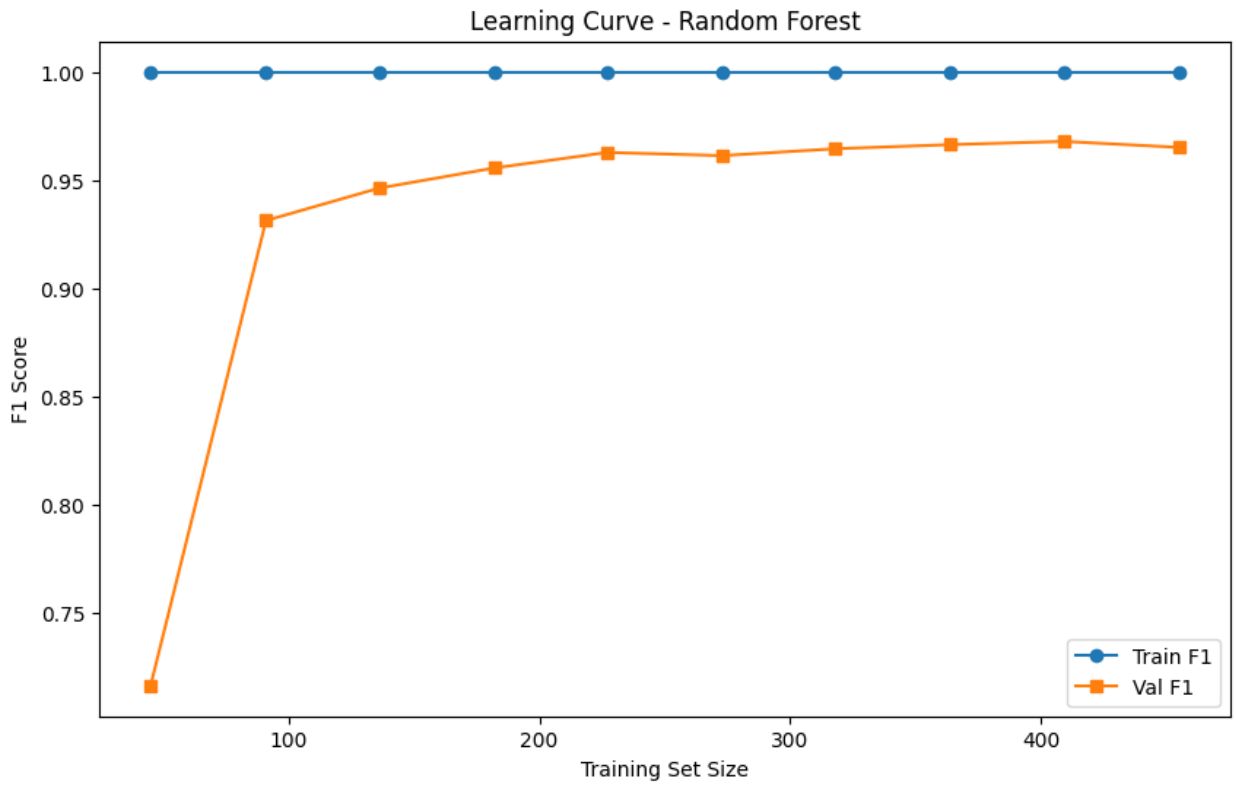
15.5 ROC Curves

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve, auc

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_tr_sc = scaler.fit_transform(X_tr)
X_te_sc = scaler.transform(X_te)

fig, ax = plt.subplots(figsize=(10, 7))

for name, model in models.items():
    if hasattr(model, 'predict_proba'):
        model.fit(X_tr_sc, y_tr)
        proba = model.predict_proba(X_te_sc)[:, 1]
    else:
        model.fit(X_tr_sc, y_tr)
        proba = model.decision_function(X_te_sc)
    fpr, tpr, _ = roc_curve(y_te, proba)
    roc_auc = auc(fpr, tpr)
    ax.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc:.3f})')

ax.plot([0,1],[0,1],'k--', label='Random')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC Curves - All Models')
ax.legend(loc='lower right')
plt.tight_layout()
plt.show()
```



15.6 Learning Curves

```
from sklearn.model_selection import learning_curve
import numpy as np

pipe_rf = Pipeline([('scaler', StandardScaler()),
                    ('clf',
RandomForestClassifier(random_state=42))])

train_sizes, train_scores, val_scores = learning_curve(
    pipe_rf, X, y, cv=5,
    train_sizes=np.linspace(0.1, 1.0, 10),
    scoring='f1', n_jobs=-1)

plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-',
label='Train F1')
plt.plot(train_sizes, val_scores.mean(axis=1), 's-', label='Val
F1')
plt.xlabel('Training Set Size')
plt.ylabel('F1 Score')
plt.title('Learning Curve - Random Forest')
plt.legend()
plt.show()
```



15.7 Final Comparison Table

Algorithm	Train Time	Interpretability	Scalability	Best For
Decision Tree	Fast	High	Medium	Interpretable rules
Naive Bayes	Very Fast	Medium	High	Text, categorical
KNN	None (lazy)	Low	Low	Small datasets
SVM	Moderate	Low	Medium	High-dim, small n
Random Forest	Moderate	Medium	High	General purpose
Grad. Boosting	Slow	Low	Medium	Tabular competitions
Logistic Reg.	Fast	High	High	Linear, probabilistic

15.8 Lab Exercises

1. Run the model comparison script (Section 15.4) on the Breast Cancer dataset. Produce the boxplot and identify the top 2 models by median F1.

```
import numpy as np
from scipy.stats import friedmanchisquare

# Identifying top 2 by median F1 from existing df_results
medians = df_results.median().sort_values(ascending=False)
top_2 = medians.head(2)
print("Top 2 Models by Median F1 Score:")
print(top_2)

# Identify highest AUC from existing ROC plots (numerical check)
auc_results = {}
for name, model in models.items():
    if hasattr(model, 'predict_proba'):
        proba = model.predict_proba(X_te_sc)[:, 1]
    else:
        proba = model.decision_function(X_te_sc)
    fpr, tpr, _ = roc_curve(y_te, proba)
    auc_results[name] = auc(fpr, tpr)

best_auc_model = max(auc_results, key=auc_results.get)
print(f"\nHighest AUC achieved by: {best_auc_model}
({auc_results[best_auc_model]:.4f})")
```

```
Top 2 Models by Median F1 Score:
Logistic Regression    0.985915
SVM                    0.979444
dtype: float64

Highest AUC achieved by: Naive Bayes (0.9974)
```

2. Plot ROC curves for all 7 models. Which achieves the highest AUC?

```
titanic_results = {}
# Using X_tit and y_tit defined earlier in the notebook
for name, model in models.items():
    pipe = Pipeline([('scaler', StandardScaler()), ('clf', model)])
    scores = cross_val_score(pipe, X_tit, y_tit, cv=10,
scoring='accuracy', n_jobs=-1)
    titanic_results[name] = scores.mean()

ranked_titanic =
pd.Series(titanic_results).sort_values(ascending=False).to_frame('Mean
Accuracy')
print("Ranked Models on Titanic Dataset:")
display(ranked_titanic)
```

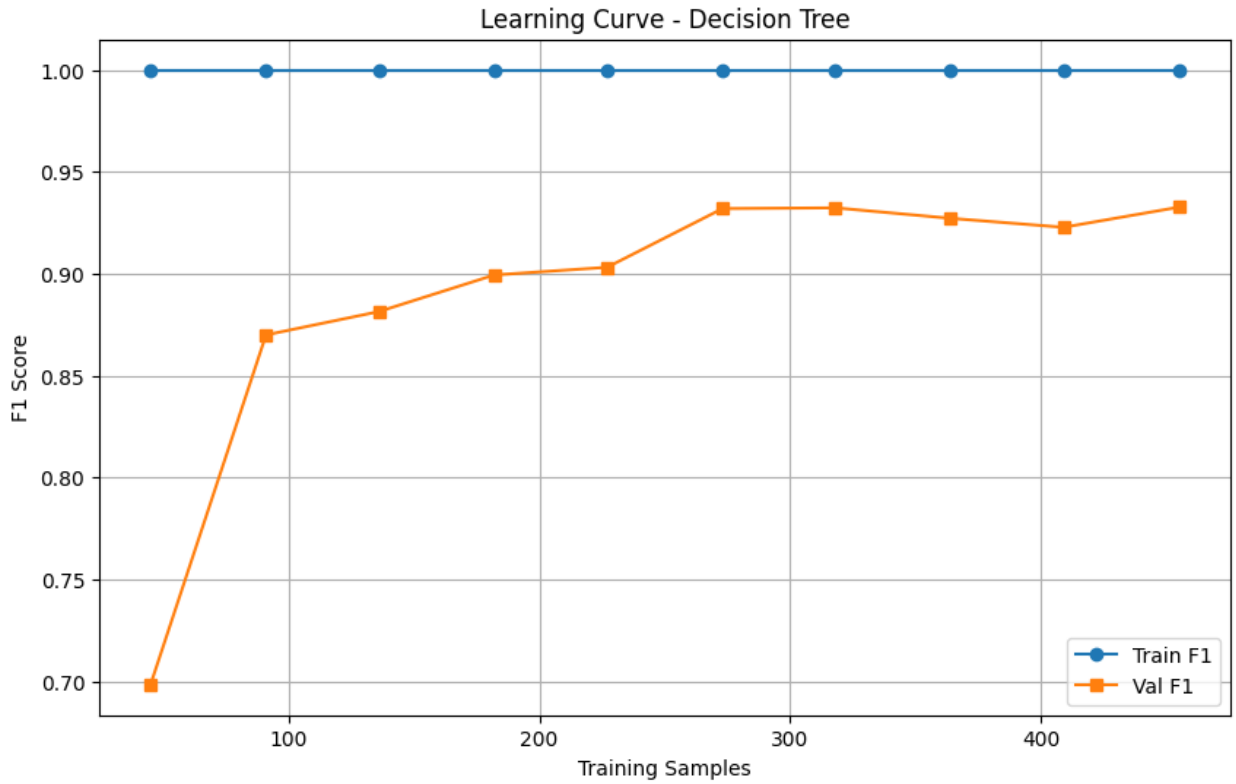
Ranked Models on Titanic Dataset:

	Mean Accuracy
Gradient Boosting	0.837291
SVM	0.824931
Random Forest	0.810437
KNN	0.808165
Logistic Regression	0.789001
Naive Bayes	0.786779
Decision Tree	0.772222

3. For the Titanic dataset, compare all 7 models using 10-fold CV accuracy. Report results in a ranked table.
4. Plot a learning curve for the Decision Tree. Does the curve suggest underfitting or overfitting?

```
pipe_dt = Pipeline([('scaler', StandardScaler()), ('clf',
DecisionTreeClassifier(random_state=42))])

train_sizes, train_scores, val_scores = learning_curve(
```

```
    pipe_dt, X, y, cv=5, train_sizes=np.linspace(0.1, 1.0, 10),
scoring='f1', n_jobs=-1)

plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', label='Train
F1')
plt.plot(train_sizes, val_scores.mean(axis=1), 's-', label='Val F1')
plt.title('Learning Curve - Decision Tree')
plt.xlabel('Training Samples')
plt.ylabel('F1 Score')
plt.legend()
plt.grid(True)
plt.show()

print("Observation: Large gap between curves usually indicates
overfitting, while both being low indicates underfitting.")
```



5. Apply Friedman test (scipy.stats.friedmanchisquare) to determine if differences in CV scores are statistically significant.

```
# Using the 10-fold CV results from the Breast Cancer dataset
(df_results)
stat, p = friedmanchisquare(*[df_results[model] for model in
models.keys()])
```

```
print(f"Friedman Test Statistic: {stat:.4f}")
print(f"P-value: {p:.4f}")

if p < 0.05:
    print("\nResult: The differences in CV scores are statistically
significant (p < 0.05).")
else:
    print("\nResult: The differences in CV scores are NOT statistically
significant (p >= 0.05).")
```

```
Friedman Test Statistic: 26.9234
P-value: 0.0001

Result: The differences in CV scores are statistically significant (p < 0.05).
```

15.9 Summary

Rigorous model comparison requires consistent experimental protocols: the same data splits, the same cross-validation strategy, and appropriate evaluation metrics. No single metric tells the full story - precision, recall, F1, and AUC should all be examined in context. Learning curves diagnose overfitting and underfitting, guiding decisions on data collection and model complexity.